Cheatsheet · 29 chapters on one page

GPU Programming cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

147 key ideas | 169 formulas | 53 tables | 57 tips and traps

STEP 1 Why GPUs

1.1 Why GPUs: latency, throughput and power

CPU against GPU design, latency and throughput, FPS and batch time, clock and power, peak FLOPs, and the hardware spectrum.

KEY IDEAS

- **Two designs** CPU: big control and caches, few strong ALUs, low latency. GPU: thousands of small ALUs, small control, high throughput.
- **Latency and throughput** Latency = time for one job. Throughput = jobs ÷ time. FPS = 1000 ÷ ms per frame. Batch time = ⌈N ÷ C⌉ × time per round.
- **Power** $P \propto \alpha\, C\, V^2\, f$. Higher clock needs higher voltage, so power explodes. GPUs use many cores at a moderate clock.
- **Peak** Peak = FLOPs per tick per core × clock × cores. FMA = 2 FLOPs. T4 ≈ 8.1, A100 FP32 ≈ 19.5 TFLOP/s.
- **Hardware spectrum** CPU → GPU → FPGA → ASIC: less flexible, more efficient. TPU and NPU are ASIC style AI accelerators. The CPU is always the host.

FORMULAS

Latency and throughput — Exam favourite

Latency = time to finish ONE job

Throughput = jobs finished ÷ time taken

A CPU is built for low latency. A GPU is built for high throughput.

Lesson: Latency, throughput and frames per second

Frames per second

FPS = 1000 ÷ (ms per frame)

ms per frame = 1000 ÷ FPS

There are 1000 milliseconds in one second.

Lesson: Latency, throughput and frames per second

Time for a batch — Exam favourite

$$T = \lceil N \div C \rceil \times (\text{time per round})$$

N jobs, C jobs done together in one round. ⌈ ⌉ means round up, because a part full round still costs a whole round.

Lesson: Latency, throughput and frames per second

Pipeline

latency = sum of stage times

throughput = 1 ÷ (slowest stage)

Once the line is full, one result comes out every time the slowest stage finishes.

Lesson: Latency, throughput and frames per second

Cycle time

$$t_{\text{cycle}} = 1 \div f$$

1 GHz = 10^9 ticks per second, so one tick lasts 1 ns. At 800 MHz a tick lasts 1.25 ns.

Lesson: Clock speed, power and peak work

Dynamic power — Exam favourite

$$P_{\text{dyn}} \propto \alpha \cdot C \cdot V^2 \cdot f$$

α is how often the circuit switches, C its capacitance, V the voltage (squared), f the clock. A faster clock needs a higher voltage, so power grows much faster than speed.

Lesson: Clock speed, power and peak work

Peak work per second Exam favourite

$$\text{peak} = (\text{FLOPs per tick per core}) \times f \times \text{cores}$$

One FMA, a × b + c, counts as 2 FLOPs.

Lesson: Clock speed, power and peak work

Host and device

Host = CPU (starts the program, owns the operating system)

Device = GPU (runs the parallel parts you send it)

Every GPU system is driven by a CPU. CUDA is the library and language extension that lets the host hand work to the device.

Lesson: CPU, GPU, TPU, NPU, FPGA and ASIC

TABLES

Table: part of the chip · cpu · gpu

Part of the chip	CPU	GPU
Control logic (decides what runs next)	Large and clever: branch prediction, out of order execution	Small and simple, shared by a group of threads
Cache (fast memory on the chip)	Large per core	Small per core, plus a shared L2
ALUs (the units that add and multiply)	A few, powerful	Thousands, small and energy efficient
Clock	High, about 2 to 5 GHz	Moderate, about 0.8 to 2 GHz
Built for	Low latency: one task, fast	High throughput: many tasks per second

Lesson: Two ways to build a processor

Table: platform · strength · weakness · examples from the lecture

Platform	Strength	Weakness	Examples from the lecture
CPU	Most flexible, cheapest, runs any program	Limited parallelism	Arm Cortex-M, Raspberry Pi
GPU	Massive parallelism, good programmability	Power hungry	NVIDIA Jetson, T4, A100, Arm Mali
FPGA	Custom circuits, low latency, good work per watt, can be reprogrammed	Small on chip memory, needs hardware design skill	Xilinx Zynq, Intel Arria 10
ASIC (TPU, NPU)	Lowest power, highest performance for one task	Very costly to design, long time to market, fixed	Google Edge TPU, Ascend 310

Lesson: CPU, GPU, TPU, NPU, FPGA and ASIC

TIPS AND TRAPS

COMMON MISTAKE GPU does not mean "always faster". A GPU wins only when there is enough independent work to keep thousands of small units busy. For a single small task the CPU usually wins.

Lesson: Two ways to build a processor

REMEMBER Peak FLOP/s is a ceiling, not a promise. Real kernels usually reach a fraction of it, because they wait for memory. Chapter 4.2 (the roofline) shows how to predict that fraction.

Lesson: Clock speed, power and peak work

IN THE EXAM A GPU never runs alone. The CPU boots the machine, runs the serial parts and launches GPU work. This is why the time to copy data between them matters so much (chapter 1.2).

Lesson: CPU, GPU, TPU, NPU, FPGA and ASIC

1.2 Speedup, the PCIe trip and Amdahl's law

Speedup with copies counted, Amdahl's law and its ceiling, working backwards, Gustafson's law, and break even sizes.

KEY IDEAS

- **Speedup** S = old ÷ new. Count copy in + kernel + copy out. Copy time = bytes ÷ bandwidth (PCIe about 12 to 16 GB/s).
- **Amdahl** S(N) = 1 ÷ ((1 − P) + P/N). Ceiling 1 ÷ (1 − P). Efficiency S ÷ N. Back: P = (1 − 1/S) ÷ (1 − 1/N).
- **Part speedup** Speeding a part p by s gives 1 ÷ ((1 − p) + p/s). Speed up the biggest part first.
- **Gustafson** Scaled speedup N − (1 − P)(N − 1) when the problem grows with N (weak scaling).
- **Keep data on the GPU** Copy once, run many kernels, copy back once. Small jobs have a break even size below which the CPU wins.

FORMULAS

Speedup — Exam favourite

$$S = \text{old time} \div \text{new time}$$

S = 10 means the new version is ten times faster. With FPS, use new FPS ÷ old FPS.

Lesson: Speedup and the trip over PCIe

Total GPU time — Exam favourite

$$T_{\text{GPU}} = T_{\text{copy in}} + T_{\text{kernel}} + T_{\text{copy out}}$$

The GPU is worth it only if $T_{\text{GPU}} < T_{\text{CPU}}$.

Lesson: Speedup and the trip over PCIe

Copy time

$$T_{\text{copy}} = \text{bytes} \div \text{bandwidth}$$

PCIe 3.0 x16 gives about 12 to 16 GB/s in practice. 12 GB/s = 12×10^9 bytes per second.

Lesson: Speedup and the trip over PCIe

Amdahl's law — Exam favourite

$$S(N) = 1 \div [\ (1 - P) + P \div N\ ]$$

P is the parallel fraction measured on one processor, N the number of processors.

Lesson: Amdahl's law

The ceiling — Exam favourite

$$S_{\max} = 1 \div (1 - P)$$

Let N grow without limit. 95 percent parallel can never beat 20 times.

Lesson: Amdahl's law

Working backwards

$$P = (1 - 1/S) \div (1 - 1/N)$$

From a measured speedup S on N processors.

Lesson: Amdahl's law

Efficiency

$$E = S \div N$$

How much of each processor is really used.

Lesson: Amdahl's law

Speeding up one part s times

$$S = 1 \div [\ (1 - p) + p \div s\ ]$$

The same law for any part p that gets s times faster, for example a kernel.

Lesson: Amdahl's law

Gustafson's law (scaled speedup) — Exam favourite

$$S_{\text{scaled}} = N - (1 - P)(N - 1)$$

Here P is the parallel share of the time measured on the N processor run. The speedup grows almost in a straight line with N.

Lesson: Gustafson's law and scaling (beyond the lecture)

TABLES

Table: grid · cpu (ms) · gpu (ms) · shared gpu (ms)

Grid	CPU (ms)	GPU (ms)	Shared GPU (ms)	GPU speedup
256 × 256	51.55	0.48	0.53	107.76
512 × 512	236.54	1.48	1.87	160.28
1024 × 1024	906.35	5.34	6.13	169.79
2048 × 2048	3593.75	17.75	16.32	202.44

Lesson: Speedup and the trip over PCIe

Table: · strong scaling (amdahl) · weak scaling (gustafson)

	Strong scaling (Amdahl)	Weak scaling (Gustafson)
What is fixed	Total problem size	Work per processor
Question asked	How much faster does the same job finish?	How much bigger a job fits in the same time?
Limit	1 ÷ (1 − P)	Grows with N
GPU example	One 1024 × 1024 grid on more SMs	Grid grows from 1024^2 to 4096^2 with a bigger GPU

Lesson: Gustafson's law and scaling (beyond the lecture)

TIPS AND TRAPS

COMMON MISTAKE Keep data on the GPU across many kernel launches. Copy in once, run all time steps, copy out once. Copying every step can turn a 158 times speedup into 12 times (workbook P1.3.5).

Lesson: Speedup and the trip over PCIe

COMMON MISTAKE P is a fraction of the time on one processor, not a fraction of the lines of code. A tiny loop that runs for 90 percent of the time gives P = 0.9.

Lesson: Amdahl's law

IN THE EXAM In an exam, Amdahl is the default unless the question says the problem grows with the number of processors.

Lesson: Gustafson's law and scaling (beyond the lecture)

1.3 First GPU workloads: heat simulation and convolution

The heat simulation update rule and its stability, why it suits a GPU, and normal against depthwise separable convolution cost.

KEY IDEAS

- **Heat rule** T_new = T + α(up + down + left + right − 4T) + βQ. 1-D uses −2T. Updates = cells × steps.
- **Why parallel** Each cell needs only old neighbour values, so one thread per cell. Keep two buffers (old and new).
- **Stability** Explicit scheme: α ≤ 1/2 in 1-D, 1/4 in 2-D, 1/6 in 3-D, or values blow up.
- **Convolution cost** Normal H W Cin Cout K^2. Separable H W Cin K^2 + H W Cin Cout. Ratio (K^2 + Cout) ÷ (K^2 Cout) → $1/K^2$.
- **Same pattern** Heat updates and convolutions are stencils: each output reads a small neighbourhood. Tiling with halos speeds them up later.

FORMULAS

The heat equation

$$\partial T/\partial t = \alpha\,(\partial^2 T/\partial x^2 + \partial^2 T/\partial y^2) + Q$$

T is temperature, α the thermal diffusivity (how fast heat spreads), Q the heat the battery makes itself.

Lesson: Workload 1: a battery heat simulation

2-D update rule (5 points) Exam favourite

$$T^{\text{new}} = T + \alpha\,(\text{up} + \text{down} + \text{left} + \text{right} - 4T) + \beta Q$$

New temperature = old temperature + heat from the four neighbours + heat made inside.

Lesson: Workload 1: a battery heat simulation

1-D version

$$T^{\text{new}} = T + \alpha\,(\text{left} + \text{right} - 2T) + \beta Q$$

The same idea along a rod.

Lesson: Workload 1: a battery heat simulation

How much work

$$\text{cells} = \text{height} \times \text{width}$$

$$\text{updates} = \text{cells} \times \text{time steps}$$

A 1024 × 1024 grid has 1,048,576 cells per step.

Lesson: Workload 1: a battery heat simulation

Normal convolution Exam favourite

$$\text{mults} = H \times W \times C_{\text{in}} \times C_{\text{out}} \times K \times K$$

Every output pixel of every output channel looks at a K × K patch of every input channel.

Lesson: Workload 2: image recognition and convolution cost

Depthwise separable

$$\text{depthwise} = H\,W\,C_{\text{in}}\,K^2$$

$$\text{pointwise} = H\,W\,C_{\text{in}}\,C_{\text{out}}$$

First one K × K filter per input channel, then a 1 × 1 convolution that mixes channels. MobileNet uses this.

Lesson: Workload 2: image recognition and convolution cost

Saving ratio Exam favourite

$$\text{separable} \div \text{normal} = (K^2 + C_{\text{out}}) \div (K^2 \times C_{\text{out}})$$

For many output channels this tends to 1 ÷ K², so at most 9 times fewer for 3 × 3 filters.

Lesson: Workload 2: image recognition and convolution cost

TIPS AND TRAPS

COMMON MISTAKE A GPU heat code must keep two buffers. If threads wrote into the same grid they read from, some would read a neighbour's new value and some its old value, and the answer would depend on timing.

Lesson: Workload 1: a battery heat simulation

REMEMBER Fewer multiplications does not always mean proportionally faster on a GPU. Depthwise layers do little work per byte loaded, so they are often limited by memory, not maths (chapter 4.2).

Lesson: Workload 2: image recognition and convolution cost

STEP 2 The CUDA programming model

2.1 Your first CUDA program

Host and device, the CUDA recipe, __global__ void kernels, vector add line by line, nvcc, asynchronous launches, error checks and unified memory.

KEY IDEAS

- **Host and device** Host = CPU + RAM, device = GPU + its DRAM. Separate memories, so copy with cudaMemcpy (or use unified memory).

- **Recipe** cudaMalloc, cudaMemcpy H2D, kernel<<<blocks, threads>>>, cudaMemcpy D2H, cudaFree. blocks = (n + B − 1) / B.
- **Qualifiers** __global__ void kernel (host launches), __device__ helper (device only), __host__ default. Kernels cannot return values.
- **Guard** if (i < n) because the last block can overhang the array.
- **Asynchronous** Launch returns at once. cudaMemcpy and cudaDeviceSynchronize wait. Check errors with cudaGetLastError after a launch.

TABLES

Table: qualifier · runs on · called from · notes

Qualifier	Runs on	Called from	Notes
__global__	device	host (and device with dynamic parallelism)	A kernel. Must return void. Launched with <<< >>>
__device__	device	device	A helper function used inside kernels. May return any type
__host__	host	host	The default for normal functions
__host__ __device__	both	both	Compiled twice, handy for small maths helpers

Lesson: Host, device and the recipe every program follows

TIPS AND TRAPS

IN THE EXAM Quiz 1 question 6: the correct kernel signature is __global__ void. A kernel cannot return a value, because thousands of threads run it and the launch returns to the host at once without waiting. Results go back through memory. __shared__ is a memory space, not a function qualifier.

Lesson: Host, device and the recipe every program follows

COMMON MISTAKE The order of the six lines is not guaranteed. Blocks may run in any order, so block 1 can print first. Never write code that depends on the order blocks run in.

Lesson: Hello world, asynchronous launches and unified memory

2.2 Threads, blocks and grids: indexing in 1-D, 2-D and 3-D

The global index, blocks and idle threads, dim3 and 2-D image indexing, going backwards, grid stride loops and matrix vector per row.

KEY IDEAS

- **Global index** i = blockIdx.x × blockDim.x + threadIdx.x. Blocks = (N + B − 1)/B. Guard i < N. Idle = blocks × B − N.
- **2-D** col = bx·Bx + tx, row = by·By + ty, index = row × width + col. Grid = (⌈W/Bx⌉, ⌈H/By⌉), guard both.
- **Linear order** thread = tx + ty·Bx + tz·Bx·By (x fastest). Warps are cut from this order.
- **Backwards** block = ⌊g ÷ per block⌋, leftover = g mod per block, x = leftover mod Bx, y = ⌊leftover ÷ Bx⌋.
- **Grid stride** Start at i = idx, keep going while i < n, and add blockDim.x × gridDim.x each time. A fixed grid then covers any N.

FORMULAS

Global index

Exam favourite

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

Skip all the full blocks before mine, then step to my seat in my block.

Lesson: The global index in one dimension

Blocks needed

Exam favourite

$$\text{blocks} = \lceil N \div B \rceil = (N + B - 1) \div B$$

Whole number division drops the remainder, so adding B − 1 first rounds up.

Lesson: The global index in one dimension

Going backwards

$$\text{block} = \lfloor i \div B \rfloor, \ \text{thread} = i - \text{block} \times B$$

Which thread handles element i.

Lesson: The global index in one dimension

Idle threads

$$\text{idle} = \text{blocks} \times B - N$$

They fail the guard and do nothing.

Lesson: The global index in one dimension

Pixels in an image

Exam favourite

$$\text{col} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

$$\text{row} = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}$$

$$\text{memory position} = \text{row} \times \text{width} + \text{col}$$

Images are stored row by row (row major). x is the column, y is the row.

Lesson: dim3, images and two dimensional grids

Grid for a W × H image

$$\text{grid} = (\lceil W \div B_x \rceil, \lceil H \div B_y \rceil)$$

Idle threads = all threads − W × H.

Lesson: dim3, images and two dimensional grids

Position to one number (x fastest)

Exam favourite

$$\text{thread} = t_x + t_y B_x + t_z B_x B_y$$

$$\text{block} = b_x + b_y G_x + b_z G_x G_y$$

$$\text{global} = \text{block} \times (\text{threads per block}) + \text{thread}$$

This is how the hardware lines threads up before cutting them into warps.

Lesson: dim3, images and two dimensional grids

One number to position

$$\text{block} = \lfloor g \div \text{threads per block} \rfloor, \ \text{leftover} = g - \text{block} \times \text{threads per block}$$

$$x = \text{leftover} \bmod B_x, \ y = \lfloor \text{leftover} \div B_x \rfloor$$

The same mod and divide works for the block inside the grid.

Lesson: Three dimensions, going backwards, and the slide correction

TABLES

Table: variable · meaning · range

Variable	Meaning	Range
threadIdx.x/y/z	my position inside my block	0 to blockDim − 1
blockIdx.x/y/z	my block's position inside the grid	0 to gridDim − 1
blockDim.x/y/z	size of every block	set by the launch, at most 1024 threads in total
gridDim.x/y/z	number of blocks	x up to $2^{31} - 1$, y and z up to 65,535

Lesson: The global index in one dimension

TIPS AND TRAPS

REMEMBER — Using 32 × 8 blocks for 1920 × 1080 wastes nothing (1080 ÷ 8 = 135) and each warp covers one full 32 pixel row. Picking a block shape that divides the image is free speed.

Lesson: dim3, images and two dimensional grids

IN THE EXAM — Exams love going backwards. Always divide by the number of threads per block first, then split the leftover with mod and divide, x first.

Lesson: Three dimensions, going backwards, and the slide correction

COMMON MISTAKE In matVecMul neighbouring threads read A[i × N + j] for the same j, which are N floats apart. That is a strided, uncoalesced read (chapter 3.2). A transposed layout or one warp per row is faster for large N.

Lesson: Grid stride loops and one thread per row

2.3 SMs, warps and hiding latency

SM anatomy, warps and lanes, wasted lanes, Little's law and latency hiding, and registers per thread.

KEY IDEAS

- **Mapping** Thread → core lane, warp → scheduler, block → one SM for life, grid → whole GPU.
- **Warps** warps = ⌈B/32⌉, warp = ⌊t/32⌋, lane = t mod 32, t = x + y·Bx. Efficiency B ÷ (32 × warps).
- **Little's law** In flight = latency × throughput. Warps = that ÷ 32. Bytes in flight = bandwidth × latency.
- **Latency hiding** Zero cost warp switching because registers stay on chip. Need many warps or independent instructions.
- **Registers** Registers per thread = file ÷ threads. 65,536 on modern SMs. Too many spill to slow local memory.

FORMULAS

Warps per block Exam favourite

$$\text{warps} = \lceil B \div 32 \rceil$$

A part full warp still takes 32 lanes.

Lesson: Warps, lanes and SIMT

Warp and lane of thread t Exam favourite

$$\text{warp} = \lfloor t \div 32 \rfloor, \ \text{lane} = t - 32 \times \text{warp}$$

For 2-D blocks first compute $t = x + y \times B_x$.

Lesson: Warps, lanes and SIMT

Wasted lanes

$$\text{wasted} = 32 \times \text{warps} - B$$

$$\text{efficiency} = B \div (32 \times \text{warps})$$

Use block sizes that are multiples of 32.

Lesson: Warps, lanes and SIMT

Little's law Exam favourite

$$\text{jobs in flight} = \text{latency (ticks)} \times \text{throughput (jobs per tick)}$$

To keep a pipe full you need as many jobs inside it as its length times its width.

Lesson: Hiding latency: Little's law

In warps

$$\text{warps in flight} = (\text{latency} \times \text{operations per tick}) \div 32$$

One warp instruction is 32 operations.

Lesson: Hiding latency: Little's law

Memory version

$$\text{bytes in flight} = \text{bandwidth} \times \text{latency}$$

To use all the bandwidth, this many bytes must be on the way at every moment.

Lesson: Hiding latency: Little's law

Registers per thread

$$\text{registers per thread} = \text{registers on the SM} \div \text{threads on the SM}$$

Lecture example: 32K registers for 1024 threads gives 32 each.

Lesson: Registers: the hidden limit

Binary K

$$32\text{K} = 32 \times 1024 = 32{,}768$$

$$64\text{K} = 65{,}536$$

For registers K means 1024.

Lesson: Registers: the hidden limit

Bits to name a register

$$\text{smallest } b \text{ with } 2^b \geq \text{number of registers}$$

An instruction names its operands, so 32 registers need 5 bits, 255 need 8.

Lesson: Registers: the hidden limit

TABLES

Table: software (what you write) · hardware (what runs it) · rule

Software (what you write)	Hardware (what runs it)	Rule
Thread	A CUDA core (one lane)	Has its own registers and program counter
Warp of 32 threads	A warp scheduler	Issues one instruction for all 32 lanes
Block (up to 1024 threads)	One SM	Stays on that SM until it finishes, shares its shared memory
Grid (all blocks of a launch)	The whole GPU	Blocks spread over all SMs in any order

Lesson: Streaming multiprocessors: what really runs your threads

Table: fermi sm part (lecture 1) · count · job

Fermi SM part (lecture 1)	Count	Job
CUDA cores	32	Integer and floating point units, one lane each
Warp schedulers and dispatch units	2	Two warps can issue at the same time
Register file	32,768 × 32 bit	Private variables of every resident thread
Load/store units	16	Addresses for 16 threads per clock
Special function units	4	sin, cos, reciprocal, square root
Shared memory / L1	64 KB	Split 48/16 or 16/48 between shared memory and L1

Lesson: Streaming multiprocessors: what really runs your threads

TIPS AND TRAPS

COMMON MISTAKE In class it was said that grids sit inside cores. It is the other way round: a thread runs on a core, a block on one SM, and a grid across all SMs.

Lesson: Streaming multiprocessors: what really runs your threads

REMEMBER Latency hiding needs either many warps (thread level parallelism) or several independent instructions inside each warp (instruction level parallelism). Two independent loads per warp halve the warps needed.

Lesson: Hiding latency: Little's law

2.4 Occupancy: how many warps fit on an SM

Occupancy from four limits, register rounding, shared memory reserve, launch bounds and the occupancy API.

KEY IDEAS

- **Occupancy** Active warps ÷ max warps per SM. Blocks = min(warp limit, block limit, register limit, shared memory limit).
- **Block limit** Small blocks hit the block cap. Smallest block for 100 percent = max threads ÷ max blocks (A100: 64).
- **Registers** Per warp = ⌈32r ÷ 256⌉ × 256. Blocks = ⌊⌊65,536 ÷ per warp⌋ ÷ warps per block⌋.
- **Shared memory** Blocks = ⌊pool ÷ (bytes + 1 KB reserved)⌋ on CC 8.x.
- **Not the goal** Enough warps to hide latency is enough. Measure achieved occupancy in Nsight Compute.

FORMULAS

Occupancy — Exam favourite

$$\text{occupancy} = \text{active warps per SM} \div \text{maximum warps per SM}$$

A100: maximum 64 warps (2048 threads). 48 active warps is 75 percent.

Lesson: What occupancy means

Blocks that fit — Exam favourite

$$\text{blocks} = \min(\text{warp limit}, \text{block limit}, \text{register limit}, \text{shared memory limit})$$

Every resource must fit at the same time, so the tightest one decides.

Lesson: What occupancy means

Smallest block for full occupancy

$$B_{\min} = \text{max threads per SM} \div \text{max blocks per SM}$$

A100: 2048 ÷ 32 = 64 threads. Anything smaller cannot reach 100 percent.

Lesson: Limit 1 and 2: threads and blocks

Registers per warp — Exam favourite

$$\text{per warp} = \lceil (r \times 32) \div 256 \rceil \times 256$$

r = registers per thread. 37 registers give 1184, rounded up to 1280, the same as 40.

Lesson: Limit 3: registers, with rounding

Register limit in blocks — Exam favourite

$$\text{blocks} = \lfloor \lfloor 65{,}536 \div \text{per warp} \rfloor \div \text{warps per block} \rfloor$$

First whole warps, then whole blocks.

Lesson: Limit 3: registers, with rounding

Shared memory limit — Exam favourite

$$\text{blocks} = \lfloor \text{shared memory per SM} \div (\text{bytes per block} + \text{reserved}) \rfloor$$

CC 8.6: 102,400 bytes per SM, 1024 reserved per block.

Lesson: Limit 4: shared memory

TABLES

Table: gpu (compute capability) · max threads · max warps · max blocks

GPU (compute capability)	Max threads	Max warps	Max blocks	Registers	Shared memory per SM
Fermi (2.0)	1536	48	8	32,768	48 KB
T4 (7.5)	1024	32	16	65,536	64 KB
A100 (8.0)	2048	64	32	65,536	up to 164 KB
RTX 30xx (8.6)	1536	48	16	65,536	up to 100 KB
H100 (9.0)	2048	64	32	65,536	up to 228 KB

Lesson: What occupancy means

TIPS AND TRAPS

REMEMBER — The flag -maxrregcount=N caps registers for every kernel in a file. __launch_bounds__ is better because it is per kernel and says why.

Lesson: Limit 3: registers, with rounding

REMEMBER — The printed numbers above are what this code gives on an A100 by the four limit rule. They were worked out, not measured on a real GPU.

Lesson: Limit 4: shared memory

STEP 3 GPU memory

3.1 The GPU memory hierarchy

Memory spaces, scope and lifetime, constant memory, caches, pinned and unified memory, copy time.

KEY IDEAS

- **Spaces** Register (thread), local (thread, in DRAM), shared (block), global (all, host, until cudaFree), constant (all, read only).
- **Q8** Visible to all SMs for the whole application: global memory. L2 is a cache, not a space.
- **Constant** 64 KB, host writes with cudaMemcpyToSymbol, broadcast when a warp reads one address.
- **Host memory** Pageable malloc, pinned cudaMallocHost (needed for async overlap), managed cudaMallocManaged with prefetch.
- **Copies** Copy time = bytes ÷ bandwidth. PCIe is far slower than device DRAM. Copy once, reuse many times.

FORMULAS

Copy time Exam favourite

$$t = \text{bytes} \div \text{bandwidth}$$

The PCIe link is far slower than the GPU's own DRAM, so copy once and reuse.

Lesson: Host memory: pageable, pinned and unified

Worth sending to the GPU

$$t_{\text{copy in}} + t_{\text{kernel}} + t_{\text{copy out}} < t_{\text{CPU}}$$

From chapter 1.2: count the copies too.

Lesson: Host memory: pageable, pinned and unified

TABLES

Table: memory · who sees it · lives for · speed

Memory	Who sees it	Lives for	Speed	How you get it
Registers	one thread	the thread	about 1 tick	any normal local variable
Local	one thread	the thread	DRAM speed (cached)	big private arrays, spilled registers
Shared	all threads of one block	the block	about 20 to 30 ticks	__shared__
Global	all threads, all kernels, the host	until cudaFree	about 400 to 800 ticks	cudaMalloc
Constant	all threads, read only	the application	fast when the warp reads one address	__constant__ and cudaMemcpyToSymbol

Lesson: Five places a value can live

Table: cache · where · what it holds

Cache	Where	What it holds
L1 / texture	per SM, shares space with shared memory	recent global and local loads
Constant cache	per SM	recent constant memory reads
L2	one for the whole GPU	all traffic to device DRAM (40 MB on an A100)

Lesson: Constant memory and caches

TIPS AND TRAPS

COMMON MISTAKE Local memory is not near the thread. "Local" means private. It lives in device DRAM and is only cached on chip.

Lesson: Five places a value can live

IN THE EXAM A memory space is something you name and address. A cache is a hardware copy of global data that fills and empties by itself. This difference decides questions like Q8.

Lesson: Quiz 1 question 8: visible to all SMs, for the whole application

3.2 Coalescing: reading global memory well

Sectors per warp read, stride and alignment, Quiz 1 Q5, sector model, AoS and SoA, transposes.

KEY IDEAS

- **Sectors** Warp loads are coalesced into 32 byte sectors. Efficiency = 32 × element ÷ (sectors × 32).
- **Strides** Floats: stride 1 → 4 sectors, 2 → 8, 4 → 16, ≥ 8 → 32. Misaligned start adds a sector.
- **Row major** Let threadIdx.x move along the row. Q5 store B[x·h + y] touches 32 segments.
- **Sector model** Useful bandwidth ≈ stride 1 bandwidth ÷ min(s, 8). T4 206 GB/s at stride 1.
- **Fixes** SoA instead of AoS, float4 loads, and shared memory tiles for transposes.

FORMULAS

Bytes the warp wants Exam favourite

$$\text{useful} = 32 \times \text{element size}$$

Floats: 128 bytes. Doubles: 256 bytes.

Lesson: Memory hands out 32 byte boxes

Efficiency Exam favourite

$$\text{efficiency} = \text{useful bytes} \div (\text{sectors} \times 32)$$

Stride 1 floats: 128 ÷ (4 × 32) = 100 percent.

Lesson: Memory hands out 32 byte boxes

Sectors for stride s (floats, aligned)

$$\text{sectors} = \min(32, 4 \times s)$$

Each thread lands in its own sector once s ≥ 8.

Lesson: Memory hands out 32 byte boxes

Sector model Exam favourite

$$\text{useful bandwidth} \approx (\text{stride 1 bandwidth}) \div \min(s, 8)$$

Once threads are 8 floats apart each lane needs its own sector, and the model stops falling. Real GPUs fall further at big strides because of DRAM page and TLB effects.

Lesson: The stride benchmark (lecture 11)

Vector loads

$$\text{float4: 32 lanes} \times 16 \text{ bytes} = 512 \text{ bytes} = 16 \text{ sectors, 100 percent}$$

One instruction moves four floats per thread, so fewer instructions for the same bytes. The address must be 16 byte aligned.

Lesson: Data layout: AoS against SoA, and wider loads

TABLES

Table: thread t reads · neighbouring lanes are · sectors per warp read (floats)

Thread t reads	Neighbouring lanes are	Sectors per warp read (floats)
A[row][t] (along a row)	4 bytes apart	4 (100 percent)
A[t][col] (down a column of width 1024)	4096 bytes apart	32 (12.5 percent)

Lesson: Rows, columns and Quiz 1 question 5

Table: stride · model (gb/s) · sectors per warp read

Stride	Model (GB/s)	Sectors per warp read
1	206	4
2	103	8
4	51.5	16
8	25.75	32
16	25.75	32
32	25.75	32

Lesson: The stride benchmark (lecture 11)

TIPS AND TRAPS

REMEMBER AoS is natural for the CPU and often slow on the GPU. When kernels touch only some fields, store each field in its own array.

Lesson: Data layout: AoS against SoA, and wider loads

3.3 Shared memory and bank conflicts

Shared memory use, barriers, 32 banks, gcd conflict rule, broadcast, padding and layout.

KEY IDEAS

- **Shared memory** __shared__, one copy per block, fill then __syncthreads() then read. extern __shared__ size is the third launch value.
- **Banks** 32 banks × 4 bytes. bank = ⌊byte ÷ 4⌋ mod 32. Different words in one bank take turns.
- **Conflicts** Stride s gives gcd(s, 32) ways. Same word is a broadcast, no conflict.
- **Padding** Tile [32][33] makes column reads conflict free for 128 extra bytes.
- **Layout** Let lanes write consecutive words: powers example 1024 passes against 32.

FORMULAS

Bank of an address Exam favourite

$$\text{bank} = \lfloor \text{byte address} \div 4 \rfloor \bmod 32$$

Word w lives in bank w mod 32. Byte 260 is word 65, bank 1.

Lesson: Banks and conflicts

Conflict for stride s Exam favourite

$$\text{ways} = \gcd(s, 32)$$

Odd strides are always conflict free.

Lesson: Banks and conflicts

Broadcast

$$\text{same word for many lanes} \rightarrow \text{no conflict}$$

One read is sent to every lane that asked for it.

Lesson: Banks and conflicts

TIPS AND TRAPS

COMMON MISTAKE __syncthreads() must be reached by every thread of the block. Putting it inside an if that only some threads enter can hang the block or give wrong results.

Lesson: Shared memory: a cache you control

STEP 4 Numbers and performance limits

4.1 Floating point numbers on the GPU

IEEE-754 encoding, binary fractions, float addition, non associativity, FMA and reduced precision formats.

KEY IDEAS

- **Format** value = (−1)^S × 1.f × 2^(E − bias). Float 1/8/23 bias 127, double 1/11/52 bias 1023.
- **Exact** Exact in binary only when the denominator is a power of 2. 5.6 is stored as 5.599999904632568.
- **Add** Align exponents, add mantissas, normalise, round. Not associative: compare with a tolerance.
- **FMA** a × b + c with one rounding, one instruction, counted as 2 FLOPs.
- **Formats** FP16 1/5/10 (max 65,504), BF16 1/8/7, TF32 1/8/10. Epsilon = 2^−mantissa bits.

FORMULAS

IEEE-754 value Exam favourite

$$\text{value} = (-1)^S \times 1.f \times 2^{E - \text{bias}}$$

bias = 127 for float (8 exponent bits), 1023 for double (11 bits).

Lesson: Sign, exponent and mantissa

Binary fraction

$$0.b_1b_2b_3\ldots = b_1 \times ½ + b_2 \times ¼ + b_3 \times ⅛ + \ldots$$

$1.00101_2 = 1 + ⅛ + 1/32 = 1.15625$.

Lesson: Sign, exponent and mantissa

Exact or not

a decimal fraction is exact in binary only if its denominator is a power of 2

0.625 = 5/8 is exact. 0.6 = 3/5 repeats for ever.

Lesson: Sign, exponent and mantissa

Machine epsilon

$$\varepsilon = 2^{-\text{mantissa bits}}$$

FP32: $2^{-23} \approx 1.19 \times 10^{-7}$. The gap between 1 and the next float. Relative rounding error is at most $\varepsilon \div 2$.

Lesson: Encoding by hand

TABLES

Table: format · sign · exponent · mantissa

Format	Sign	Exponent	Mantissa	About this many decimal digits
FP64 double	1	11	52	16
FP32 float	1	8	23	7
TF32 (tensor cores)	1	8	10	3
BF16	1	8	7	2 to 3
FP16 half	1	5	10	3

Lesson: Sign, exponent and mantissa

Table: exponent field · mantissa · meaning

Exponent field	Mantissa	Meaning
0	0	±0
0	not 0	subnormal (tiny numbers, no hidden 1)
1 to 254	anything	normal number
255	0	±infinity (overflow, 1 ÷ 0)
255	not 0	NaN (0 ÷ 0, √−1)

Lesson: Special values and precision on GPUs

TIPS AND TRAPS

COMMON MISTAKE The lecture 5 notes give $1.00101_2 = 1.5625$. Adding the bits gives 1 + 0.125 + 0.03125 = 1.15625 = 37/32, which is the value the workbook uses.

Lesson: Sign, exponent and mantissa

IN THE EXAM A GPU reduction adds numbers in a different order from a CPU loop, so float sums can differ in the last digits. That is expected. Compare float results with a tolerance, never with ==.

Lesson: Adding floats, and why order matters

4.2 FLOPs, bandwidth and the roofline

FLOPs and peak, bandwidth, arithmetic intensity, roofline and ridge point, reading benchmarks.

KEY IDEAS

- **FLOPs** Matmul $2n^3$, k × k filter $2k^2$ per output. Peak = SMs × cores × 2 × clock. FMA = 2 FLOPs.
- **Bandwidth** GB/s = (read + written bytes) ÷ 10^9 ÷ s. Count both directions.
- **AI** FLOPs ÷ bytes. SAXPY 1/6, plain matmul 0.25, perfect reuse n/8 float, n/16 double.
- **Roofline** Attainable = min(peak, AI × B). Ridge = peak ÷ B. Left memory bound, right compute bound.
- **Time bound** t ≥ max(FLOPs ÷ peak, bytes ÷ bandwidth). Memory bound kernels gain nothing from more cores.

FORMULAS

Matrix multiply n × n

Exam favourite

$$\text{FLOPs} = 2n^3$$

Each of n^2 outputs needs n multiplies and n adds.

Lesson: Counting the work: FLOPs

Filter k × k

$$\text{FLOPs} = 2k^2 \times \text{outputs}$$

A 5 × 5 filter on 1020 × 1020 outputs: 52,020,000.

Lesson: Counting the work: FLOPs

Peak of a GPU

Exam favourite

$$\text{peak} = \text{SMs} \times \text{cores per SM} \times 2\ (\text{FMA}) \times \text{clock}$$

A100: 108 × 64 × 2 × 1.41 GHz ≈ 19.5 TFLOP/s.

Lesson: Counting the work: FLOPs

Efficiency

$$\text{achieved} \div \text{peak}$$

A simple kernel at 3171 GFLOP/s on an H100 with about 51,000 is about 6.2 percent.

Lesson: Counting the work: FLOPs

Bandwidth

Exam favourite

$$\text{GB/s} = (\text{bytes read} + \text{bytes written}) \div 10^9 \div \text{seconds}$$

Count both directions.

Lesson: Bandwidth and arithmetic intensity

Arithmetic intensity

Exam favourite

$$\text{AI} = \text{FLOPs} \div \text{bytes moved to or from DRAM}$$

Unit FLOP per byte.

Lesson: Bandwidth and arithmetic intensity

SAXPY

$$\text{AI} = 2 \div 12 \approx 0.167$$

Read x, read y, write y (3 × 4 bytes) for one multiply and one add. DAXPY is half that.

Lesson: Bandwidth and arithmetic intensity

Matrix multiply

$$\text{no reuse: 2 FLOPs per 2 loads} \rightarrow 0.25\ (\text{float}),$$
$$\text{perfect reuse: AI} = n \div 8\ (\text{float}) \text{ or } n \div 16\ (\text{double})$$

Reuse is what raises AI.

Lesson: Bandwidth and arithmetic intensity

Attainable speed

Exam favourite

$$\text{attainable} = \min(\text{peak FLOP/s}, \text{AI} \times \text{bandwidth})$$

The lower roof wins.

Lesson: The roofline model

Ridge point

Exam favourite

$$\text{AI}_{\text{ridge}} = \text{peak} \div \text{bandwidth}$$

Left of it a kernel is memory bound, right of it compute bound.

Lesson: The roofline model

Time

$$t \geq \max(\text{FLOPs} \div \text{peak}, \text{bytes} \div \text{bandwidth})$$

The shortest possible time is set by the slower of the two.

Lesson: The roofline model

TABLES

Table: kernel · flops per element · bytes per element (float) · ai

Kernel	FLOPs per element	Bytes per element (float)	AI
y = a·x	1	8	0.125
SAXPY y = a·x + y	2	12	0.167
dot product (double)	2	16	0.125
$y = x^3 + x$	3	8	0.375

Lesson: Bandwidth and arithmetic intensity

Table: picture · cpu (ms) · gpu kernel (ms) · speedup

Picture	CPU (ms)	GPU kernel (ms)	Speedup
Lena 512 × 512	13.59	0.188	about 72
Kid	47.27	0.336	about 141

Lesson: Reading benchmark numbers (lecture 4 Sobel)

TIPS AND TRAPS

IN THE EXAM

For a memory bound kernel, doubling the maths units changes nothing. Only more bandwidth or more reuse helps. Doubling Fermi's cores gives plain matrix multiply no gain.

Lesson: The roofline model

STEP 5 Parallel patterns

5.1 Parallel reduction

Tree reduction, pairing and divergence, barrier rule, grid level reductions, atomics and warp shuffles.

KEY IDEAS

- **Tree** $\log_2 n$ levels, n − 1 additions, 1 + levels barriers. 1024 → 10 levels, 11 barriers.
- **Pairing** Sequential (t < s, s halves) packs active threads: only levels with s < 32 split a warp.
- **Barrier rule** Every thread must reach every __syncthreads(). Never put it inside a divergent if.
- **Grid** One atomicAdd per block (⌈n/B⌉ atomics) or repeated launches until one value is left.
- **Shuffles** __shfl_down_sync with offsets 16 to 1 sums a warp in 5 steps without shared memory.

FORMULAS

Levels Exam favourite

$$\text{levels} = \lceil \log_2 n \rceil$$

1024 numbers need 10 levels instead of 1023 serial additions.

Lesson: From a serial loop to a tree

Additions Exam favourite

$$\text{total additions} = n - 1$$

The tree does the same work, it only runs it in parallel.

Lesson: From a serial loop to a tree

Barriers

$$\text{barriers} = 1 \text{ (after loading)} + \text{levels}$$

A 1024 thread tree uses 11 __syncthreads().

Lesson: From a serial loop to a tree

Launches without atomics

$$\text{repeat: } n \leftarrow \lceil n \div B \rceil \text{ until } n = 1$$

1,000,000 numbers with B = 256: 3907, then 16, then 1, so 3 launches.

Lesson: Beyond one block: launches and atomics

Atomics needed Exam favourite

$$\text{one per block} = \lceil n \div B \rceil$$

2^{20} numbers with B = 1024: 1024 atomics instead of 1,048,576.

Lesson: Beyond one block: launches and atomics

TIPS AND TRAPS

REMEMBER Floating point addition is not associative (chapter 4.1), so the tree order gives a slightly different float sum from a CPU loop. Compare with a tolerance.

Lesson: Warp shuffles: the modern way

5.2 Tiled matrix multiplication

Plain against tiled loads, Quiz 1 Q2 to Q4, two barriers, ragged edges and tile limits.

KEY IDEAS

- **Plain** 2N loads per thread, $2N^3$ total (not N^2 as slide 8 says). Q2: 524,288 per block.
- **Tiled** $\lceil N/T \rceil$ phases, $2\lceil N/T \rceil$ loads per thread, $2N^3/T$ total. Q3: 32,768 per block.
- **Barriers** Sync after loading the tile and again before the next phase. Q4: without them both a and b happen (answer 3).
- **Ragged** Pad out of range loads with 0 and guard the store. n = 5, T = 4: 4 blocks, 39 idle.
- **Limits** $T^2 \le 1024$ threads so $T \le 32$. Shared memory $8T^2$ bytes. AI ≈ T/4.

FORMULAS

Loads per thread (plain) Exam favourite

$$2N$$

N from A and N from B.

Lesson: The plain kernel and its traffic

Loads per block (plain) Exam favourite

$$\text{threads per block} \times 2N$$

Quiz 1 Q2: 256 × 2048 = 524,288.

Lesson: The plain kernel and its traffic

Total loads (plain)

$$2N^3$$

Not N^2 as lecture 7 slide 8 says. For N = 1024 that is 2048 times more.

Lesson: The plain kernel and its traffic

Phases Exam favourite

$$\text{phases} = \lceil N \div T \rceil$$

N = 1024, T = 16: 64 phases.

Lesson: Tiles in shared memory

Loads per thread (tiled) Exam favourite

$$2 \times \lceil N \div T \rceil$$

One A value and one B value per phase.

Lesson: Tiles in shared memory

Total loads (tiled) Exam favourite

$$2N^2\lceil N \div T \rceil = 2N^3 \div T \text{ when T divides N}$$

Global traffic drops by a factor of T.

Lesson: Tiles in shared memory

Arithmetic intensity

$$\text{AI} \approx T \div 4 \text{ (float)}$$

Plain matmul has 0.25. T = 16 gives 4.

Lesson: Tiles in shared memory

Shared memory for two tiles

$$\text{bytes} = 2 \times T^2 \times 4$$

T = 32: 8 KB. 48 KB allows T up to 78.

Lesson: Ragged edges and tile size limits

Thread limit

$$T^2 \le 1024$$

So T ≤ 32 when each thread computes one output.

Lesson: Ragged edges and tile size limits

TIPS AND TRAPS

COMMON MISTAKE Lecture 7 slide 8 states the plain kernel reads Width² values. Each of the Width² threads reads 2 × Width values, so the total is 2 × Width³.

Lesson: The plain kernel and its traffic

5.3 Stencils, halos and heat simulation

Stencil patterns, halos, reads per output, 3-D tile intensity, heat step stability.

KEY IDEAS

- **Stencil** Each output from a fixed neighbour pattern. 2-D: ±1, ±W. 3-D: ±1, ±N, ±N².
- **Halo** Tile B + 2R. Reads per output (B + 2R)/B against 2R + 1. Barrier after loading.
- **3-D tiles** AI = 13(T − 2)³/(4T³) ≤ 3.25. Halo share 57.8 percent at T = 8. Memory bound.
- **Heat** Tnew = (1 − 6λ)T + λΣ6. Weights must add to 1 (slide 14 used 0.25 + 0.6). Stable for λ ≤ 1/6.
- **Ping-pong** Read the old grid, write the new grid, swap pointers.

FORMULAS

Neighbours in 2-D Exam favourite

$$N = i - W,\ S = i + W,\ W = i - 1,\ E = i + 1$$

Row major with width W. Pixel (row 10, col 20) of a 640 wide image is 6420.

Lesson: What a stencil is

Neighbours in 3-D Exam favourite

$$\pm 1,\ \pm N,\ \pm N^2$$

Index i = (z × N + y) × N + x.

Lesson: What a stencil is

Tile size Exam favourite

$$B + 2R \text{ values}$$

R = 3, B = 256: 262.

Lesson: Halos in shared memory

Global reads per output Exam favourite

$$(B + 2R) \div B \text{ with the tile, } 2R + 1 \text{ without}$$

R = 3, B = 256: 1.023 against 7, about 6.8 times fewer.

Lesson: Halos in shared memory

Without tiling

$$AI = 13 \div 28 \approx 0.46$$

7 multiplies and 6 adds, 7 floats loaded.

Lesson: 3-D tiles: why big tiles still stay memory bound

With T × T × T tiles Exam favourite

$$AI(T) = 13 \times (T - 2)^3 \div (4T^3)$$

Only the inner (T − 2)³ points make outputs.

Lesson: 3-D tiles: why big tiles still stay memory bound

Halo share Exam favourite

$$(T^3 - (T - 2)^3) \div T^3$$

T = 8: 57.8 percent of loaded points are border.

Lesson: 3-D tiles: why big tiles still stay memory bound

3-D explicit heat step Exam favourite

$$T_{new} = (1 - 6\lambda)\ T + \lambda \times (\text{sum of 6 neighbours})$$

λ = α Δt ÷ h². Weights add to 1, so a constant field stays constant.

Lesson: The heat equation and stability

Stability Exam favourite

$$\lambda \le 1/6 \text{ in 3-D, } 1/4 \text{ in 2-D, } 1/2 \text{ in 1-D}$$

Above it the centre weight is negative and values oscillate and grow.

Lesson: The heat equation and stability

Largest time step

$$\Delta t \le h^2 \div (6\alpha)$$

$\alpha = 0.0001$, $h = 0.01$ gives $\Delta t \le 0.167$ s.

Lesson: The heat equation and stability

TABLES

Table: stencil · points · example

Stencil	Points	Example
1-D radius R	2R + 1	moving sum, R = 3 gives 7
2-D plus (radius 1)	5	heat in a plate
2-D box radius r	$(2r + 1)^2$	blur, r = 2 gives 25
3-D plus (radius 1)	7	heat in a block
3-D box radius 1	27	smoothing a volume

Lesson: What a stencil is

TIPS AND TRAPS

COMMON MISTAKE Lecture 6 slide 14 uses 0.25 for the centre and 0.1 for each of the 6 neighbours. They add to 0.85, so a field of 100 drops to 85 every step. With neighbour weight 0.1 the centre must be $1 - 6 \times 0.1 = 0.4$.

Lesson: The heat equation and stability

5.4 Convolution and Sobel edge detection

Convolution sizes and work, Sobel, constant memory filters, halo tiles and savings.

KEY IDEAS

- **Size** out = $\lfloor (W + 2P - K)/S \rfloor + 1$. Same padding $P = (K - 1)/2$.
- **Work** MACs = $\text{out}^2\ K^2\ C_{in}\ C_{out}$. FLOPs = 2 × MACs.
- **Sobel** Gx, Gy 3 × 3. M1 = |Sx| + |Sy| clamped to 255. Weights in constant memory.
- **Tiles** Tile $(B + 2r)^2$, loads per output $(B + 2r)^2/B^2$, best saving $1 - 1/K^2$.
- **Correction** Sharpening 30 with 25, 35, 20, 40 gives 30, not 35. Smooth slopes do not change.

FORMULAS

Output size Exam favourite

$$\text{out} = \lfloor (W + 2P - K) \div S \rfloor + 1$$

W input width, K filter size, P padding, S stride.

Lesson: Convolution: slide, multiply, add

"Same" padding

$$P = (K - 1) \div 2$$

3 × 3 needs 1, 5 × 5 needs 2.

Lesson: Convolution: slide, multiply, add

Work Exam favourite

$$\text{multiply-adds} = \text{out}^2 \times K^2 \times C_{in} \times C_{out}$$

AlexNet layer 1: $55^2 \times 11^2 \times 3 \times 96 = 105{,}415{,}200$.

Lesson: Convolution: slide, multiply, add

Edge strength Exam favourite

$$M1 = |S_x| + |S_y|,\ M2 = \sqrt{(S_x^2 + S_y^2)}$$

M1 is cheaper and is what the lecture code uses. Both are clamped to 255.

Lesson: Sobel edge detection (lecture 4)

Tile side Exam favourite

$$B + 2r, \text{ with } r = (K - 1) \div 2$$

B = 32, 3 × 3: 34 × 34.

Lesson: Halo tiles for convolution

Loads per output Exam favourite

$$(B + 2r)^2 \div B^2$$

Against K^2 without a tile. B = 16, 5 × 5: 1.5625 against 25.

Lesson: Halo tiles for convolution

Saving

$$1 - (B + 2r)^2 \div (K^2 B^2)$$

Best possible for large B: $1 - 1/K^2$. 3 × 3 gives 88.9 percent.

Lesson: Halo tiles for convolution

TABLES

Table: picture · cpu · gpu kernel · speedup

Picture	CPU	GPU kernel	Speedup
Lena 512 × 512	13.59 ms	0.188 ms	about 72
Kid	47.27 ms	0.336 ms	about 141
8K frame	not given	1.65 ms (8.66 ms with copies)	copies are 81 percent

Lesson: Sobel edge detection (lecture 4)

TIPS AND TRAPS

COMMON MISTAKE Lecture 8 sharpens the pixel 30 at (2, 1) to 35. With neighbours 25, 35, 20 and 40 the result is 5 × 30 − 120 = 30. On a smooth slope sharpening changes nothing.

Lesson: Convolution: slide, multiply, add

5.5 Scan, compaction and histograms

Scans, work efficiency, compaction, histograms with privatised atomics.

KEY IDEAS

- **Scan** Inclusive y_i = x_0 + … + x_i. Exclusive starts at 0 and stops before x_i.
- **Algorithms** Hillis and Steele: $\log_2 n$ steps, $n \log_2 n - (n - 1)$ adds. Blelloch: $2 \log_2 n$ steps, $2(n - 1)$ adds.
- **Race** Never scan in place in one step. Double buffer or barrier between read and write.
- **Compaction** Exclusive scan of keep flags gives each kept value its output slot.
- **Histogram** Atomics per bin. Privatise in shared memory, merge with at most blocks × bins global atomics.

FORMULAS

Inclusive scan Exam favourite

$$y_i = x_0 + x_1 + \ldots + x_i$$

[3, 1, 7, 0] → [3, 4, 11, 11].

Lesson: Prefix sums: inclusive and exclusive

Exclusive scan Exam favourite

$$y_i = x_0 + \ldots + x_{i-1},\ y_0 = 0$$

[3, 1, 7, 0] → [0, 3, 4, 11]. The inclusive scan shifted right by one.

Lesson: Prefix sums: inclusive and exclusive

Global atomics without privatisation

one atomic per element

$n = 10^6$ bytes → 10^6 global atomics, all on 256 addresses.

Lesson: Histograms and atomic contention

Privatised (shared memory) histogram

Exam favourite

global atomics = blocks × bins at most

Each block counts in fast shared memory, then merges once.

Lesson: Histograms and atomic contention

TABLES

Table: algorithm · steps · additions · note

Algorithm	Steps	Additions	Note
Serial loop	$n - 1$	$n - 1$	one thread
Hillis and Steele	$\log_2 n$	$n \log_2 n - (n - 1)$	simple, extra work
Blelloch	$2 \log_2 n$	about $2(n - 1)$	work efficient, exclusive

Lesson: Prefix sums: inclusive and exclusive

Table: use · how the scan helps

Use	How the scan helps
Stream compaction	positions of kept items
Radix sort	positions of keys with bit 0 and bit 1
Sparse matrices (CSR)	row start offsets from row lengths
Allocating variable output	each thread's write offset

Lesson: What scans are for: compaction

TIPS AND TRAPS

COMMON MISTAKE Updating one array in place with buf[t] += buf[t – off] is a race: thread t may read buf[t – off] after thread t – off has already added to it. Double buffering or a barrier between the read and the write fixes it.

Lesson: A block scan in CUDA

STEP 6 Execution and overlap

6.1 Warp divergence and warp scheduling

Divergence cost, Quiz 1 Q9, branch free tricks, warp schedulers, hiding waits.

KEY IDEAS

- **Divergence** Inside a warp only. Time = sum of used path lengths. Efficiency = lane ticks ÷ (32 × time).
- **Q9** tid % 4 == 1 → lanes 1, 5, ..., 29, sum 120.
- **Avoid it** Branch on warp number, use selects, keep data dependent branches short.
- **Schedulers** RR waits together, GTO spreads loads (best = (instructions − 1) + L), two level in between.
- **Hiding** Warps = ⌈(W + c) ÷ c⌉. More ILP per warp means fewer warps needed.

FORMULAS

Warp time — Exam favourite

$$\text{time} = \text{sum of the lengths of every path at least one lane takes}$$

Paths no lane takes are skipped.

Lesson: Warp divergence

Efficiency — Exam favourite

$$\text{efficiency} = (\text{sum over lanes of their path length}) \div (32 \times \text{warp time})$$

8 lanes on a 10 tick path and 24 on a 30 tick path: 800 ÷ 1280 = 62.5 percent.

Lesson: Warp divergence

Worst case

$$32 \text{ different paths: } 32 \times L, \text{ efficiency } 1/32 = 3.1 \text{ percent}$$

A 32 times slowdown.

Lesson: Warp divergence

Rules (workbook 7.4)

$$\text{one instruction per tick, a load at tick t lets the warp go again at } t + L$$

finish = tick of the last load + L.

Lesson: Warp schedulers

Best possible finish — Exam favourite

$$\text{finish} \geq (\text{total instructions} - 1) + L$$

4 warps × 12 instructions, L = 12: 47 + 12 = 59. GTO reaches it.

Lesson: Warp schedulers

Warps to hide a wait — Exam favourite

$$\text{warps} = \lceil (W + c) \div c \rceil$$

Work 1 tick, wait 3: 4 warps. One warp leaves 75 percent idle, two leave 50 percent.

Lesson: How many warps hide the wait?

TABLES

Table: policy · rule · strength · weakness

Policy	Rule	Strength	Weakness
Round robin (RR)	take turns in order	fair	all warps reach their loads together and wait together
Greedy then oldest (GTO)	keep issuing the same warp until it stalls, then the oldest ready one	spreads loads out, good cache locality	less fair, a warp can starve
Two level	RR inside a small active group, switch group when all stall	between the two	needs tuning of the group size

Lesson: Warp schedulers

TIPS AND TRAPS

REMEMBER — Divergence only happens inside a warp. If whole warps take the same side, as with if ((tid / 32) % 2 == 0), there is no divergence cost at all.

Lesson: Warp divergence

REMEMBER — More independent instructions per warp (unrolling, ILP) cut the warps needed: c from 10 to 20 at W = 400 saves 20 warps.

Lesson: How many warps hide the wait?

6.2 Streams, asynchrony and overlap

Async calls, streams, overlap speedup, pinned memory, events and graphs.

KEY IDEAS

- **Async** Launches and cudaMemcpyAsync return at once. cudaMemcpy and the synchronise calls block the CPU.
- **Streams** In order inside a stream, overlap across streams. Async copies need pinned memory.

- **Pipeline** Equal stages: (N + 2)t, speedup 3N/(N + 2) < 3. Unequal: fill + N × slowest stage.
- **Danger** Not kernels overwriting each other in one stream, but the CPU reading before the GPU is done.
- **Events** cudaEventRecord marks a point. ElapsedTime for timing, StreamWaitEvent for cross stream order.

FORMULAS

Serial Exam favourite

$$t = N \times (t_{in} + t_{kernel} + t_{out})$$

One stream, nothing overlaps.

Lesson: Overlapping copies and compute

Overlapped, equal stages t Exam favourite

$$t = (N + 2) \times t, \text{ speedup } 3N \div (N + 2)$$

N = 4: 2 times. N = 16: 2.67. Never more than 3 (the number of stages).

Lesson: Overlapping copies and compute

Unequal stages

$$t \approx \text{fill} + N \times \text{slowest stage}$$

The slowest stage sets the pace. 2, 5, 2 ms with 10 chunks: 2 + 50 + 2 = 54 ms.

Lesson: Overlapping copies and compute

TABLES

Table: call · blocks the cpu? · notes

Call	Blocks the CPU?	Notes
kernel<<>>()	no	queued in the default stream
cudaMemcpy	yes	starts after earlier work in the stream, returns when the copy is done
cudaMemcpyAsync	no	needs pinned host memory to really overlap
cudaStreamSynchronize(s)	yes	until stream s is empty
cudaDeviceSynchronize()	yes	until all GPU work is done

Lesson: Who waits for whom

TIPS AND TRAPS

COMMON MISTAKE In class it was said that asynchrony is dangerous because a second kernel could overwrite the first. Work in one stream always runs in order. The real danger is the CPU reading or reusing a host buffer before the GPU has finished with it. Synchronise before reading results.

Lesson: Who waits for whom

STEP 7 Synchronisation

7.1 Race conditions and locks

Races, the three lock rules, Dekker, Peterson, Bakery and Filter.

KEY IDEAS

- **Three steps** counter++ is load, add, store. Other threads can come between them.
- **Counting orders** $(n \times s)! \div (s!)^n$. Two threads: 20, three threads: 1680. Only the serial orders are correct.
- **Three rules** Mutual exclusion, progress, bounded waiting. Breaking them gives two inside, deadlock, starvation.
- **Dekker attempts** 1 fails progress (strict turns), 2 fails mutual exclusion (not deadlock), 3 deadlocks.
- **Peterson** flag[me] = 1, turn = other, wait while flag[other] and turn == other. Correct, with fences on real hardware.
- **N threads** Bakery: tickets plus id, first come first served. Filter: N − 1 gates, starvation free but not bounded.

FORMULAS

Orders of n threads with s steps each

Exam favourite

$$(n \times s)! \div (s!)^n$$

Pick which thread takes each time slot. Steps inside one thread keep their order.

Lesson: Why counter = counter + 1 goes wrong

Two threads, three steps

$$6! \div (3! \times 3!) = 20$$

Only the 2 fully serial orders give 2. That is 10 percent.

Lesson: Why counter = counter + 1 goes wrong

Three threads, three steps

Exam favourite

$$9! \div (3!)^3 = 1680$$

Only 3! = 6 serial orders give 3. That is about 0.4 percent.

Lesson: Why counter = counter + 1 goes wrong

TABLES

Table: time · thread 1 · thread 2 · counter in memory

Time	Thread 1	Thread 2	counter in memory
1	load, r1 = 0		0
2		load, r2 = 0	0
3	r1 = 1		0
4		r2 = 1	0
5	store 1		1
6		store 1	1, but it should be 2

Lesson: Why counter = counter + 1 goes wrong

Table: rule · meaning · if it breaks

Rule	Meaning	If it breaks
Mutual exclusion	At most one thread is inside at any moment.	Two threads change the data together.
Progress	If the section is free and someone wants in, someone gets in. A thread that does not want in cannot block others.	Deadlock, or waiting for a thread that has gone home.
Bounded waiting	After I ask, others can go first only a limited number of times.	Starvation: I wait for ever.

Lesson: The critical section and Dekker's three attempts

Table: attempt · idea · mutual exclusion · progress

Attempt	Idea	Mutual exclusion	Progress	What goes wrong
1. Take turns	a shared turn variable	yes	no	Strict alternation. If thread 2 stops wanting in, thread 1 enters once and then waits for ever.
2. Check, then set	look at the other flag, then raise mine	no	yes	Both check before either sets. Both enter.
3. Set, then check	raise my flag, then look	yes	no	Both raise flags, then each waits for the other. Deadlock.

Lesson: The critical section and Dekker's three attempts

Table: lock · threads · fairness · cost per entry

Lock	Threads	Fairness	Cost per entry
Peterson	2	waits at most one turn	constant
Bakery	N	first come, first served	O(N), tickets grow without limit
Filter	N	starvation free, but no fixed bound	$O(N^2)$ checks in the worst case

Lesson: Locks for N threads: Bakery and Filter

TIPS AND TRAPS

COMMON MISTAKE The unsafe result is not guaranteed to be 1. It depends on how the hardware schedules the threads. A race means the answer is unpredictable, which is worse than always wrong.

Lesson: Why counter = counter + 1 goes wrong

IN THE EXAM Slide 8 says attempt 2 can deadlock. It cannot: a thread sets its flag only after passing its loop, so nobody waits after the flags are set. The real flaw is that both can pass the check, which breaks mutual exclusion. The class also concluded that the three attempts suffer from the same drawback. They fail in three different ways: attempt 2 checks too early, attempt 3 claims too early, attempt 1 is safe but too rigid.

Lesson: The critical section and Dekker's three attempts

IN THE EXAM In class the conclusion was that the deadlock danger is still looming. That is not correct. With reads and writes that happen in program order, Peterson meets all three rules. Swapping steps 1 and 2 (turn first, then flag) does break mutual exclusion, so the order matters.

Lesson: Peterson's algorithm and why it is correct

IN THE EXAM Slide 3 says the Filter lock has bounded waiting. It is starvation free (every waiting thread gets in in the end), but a thread can be overtaken any number of times at a gate, so there is no fixed bound r. The Bakery lock is the one with first come, first served order.

Lesson: Locks for N threads: Bakery and Filter

7.2 Barriers, atomics and GPU locks

Barriers, atomics, CAS and ticket locks, privatisation, priority locks, grid sync.

KEY IDEAS

- **Barrier against lock** __syncthreads(): all threads of one block meet. Lock: one thread at a time, only those who want it wait.
- **Who shares** Registers: nothing. Shared memory: __syncthreads(). Global across blocks: atomics, locks or a new kernel.
- **atomicCAS lock** Spin on atomicCAS(&l, 0, 1) until it returns 0, do the work, call __threadfence(), then atomicExch(&l, 0). One thread per block takes it.
- **Privatisation** Count in shared memory, then one global atomicAdd per block. 10 × 256: 2560 global atomics become 10.
- **Priority counter** Same level runs together, levels get skipped, and non resident blocks can deadlock. grid.sync() needs a cooperative launch.

FORMULAS

Compare and swap Exam favourite

$$\text{old} = *p,\ \text{if old} = \text{cmp then } *p = v,\ \text{return old}$$

If old = cmp, you won. Any other value means somebody else changed it first.

Lesson: Atomic instructions and the atomicCAS lock

Largest cooperative grid Exam favourite

$$\text{blocks} \le (\text{blocks per SM}) \times (\text{number of SMs})$$

The occupancy chapter gives blocks per SM. T4: 40 SMs, 4 blocks of 256 per SM, so at most 160 blocks.

Lesson: Priority locks and grid wide barriers

TABLES

Table: · __syncthreads() · lock from atomiccas

	__syncthreads()	Lock from atomicCAS
Purpose	everyone reaches a point	one at a time
Who waits	all threads of the block	only threads that want the lock
Scope	one block	the whole device (global memory), or shared memory
Typical use	tiled matmul, reductions, stencils	global counters, shared queues, rare updates

Lesson: Barrier or lock

Table: who shares the data · where it lives · how to synchronise

Who shares the data	Where it lives	How to synchronise
one thread	registers, local memory	nothing
threads of one block	shared memory	__syncthreads(), or shared atomics
threads of different blocks	global memory	atomics, locks, or end the kernel and start another

Lesson: Barrier or lock

Table: function · what it does as one step · returns

Function	What it does as one step	Returns
atomicAdd(p, v)	*p = *p + v	the old value
atomicSub, atomicMin, atomicMax	subtract, keep the smaller, keep the larger	the old value
atomicExch(p, v)	*p = v	the old value
atomicAnd, atomicOr, atomicXor	bit operations	the old value
atomicCAS(p, cmp, v)	if *p == cmp then *p = v	the old value

Lesson: Atomic instructions and the atomicCAS lock

Table: problem · why · fix

Problem	Why	Fix
Not mutual exclusion	All threads with the same priority pass together.	Add a real lock inside each level.
Levels get skipped	Three threads at level 2 each add 1, so the counter goes from 2 to 5.	Move the counter on once per level, when the last thread of that level is done.
Deadlock by residency	If the priority 0 blocks are not running yet because spinning blocks fill the GPU, nobody moves the counter.	Do not spin on other blocks. Use a grid barrier or separate kernels.
Starvation	A steady stream of high priority work keeps low priority work waiting.	Ageing: raise the priority of work that has waited long.

Lesson: Priority locks and grid wide barriers

TIPS AND TRAPS

COMMON MISTAKE Every thread of the block must reach every __syncthreads(). A barrier inside an if that only some threads enter is undefined behaviour: the kernel may hang, or seem to work and give wrong results. The reduction chapter shows the lecture 12 example.

Lesson: Barrier or lock

IN THE EXAM The slide's critical_kernel lets every thread call acquire_lock. Before Volta (2017) the 32 lanes of a warp move in lock step. One lane wins the lock, but the warp keeps running the spin loop for the losers and never lets the winner reach release_lock, so the kernel hangs. Newer GPUs avoid the hang, but the lock then makes every thread of the grid wait in a single line, which is very slow. Take the lock with one thread per block, and add a fence before the release.

Lesson: Atomic instructions and the atomicCAS lock

REMEMBER compute-sanitizer --tool racecheck finds races on shared memory, such as the unsafe_inc example. Run it whenever a result changes from run to run.

Lesson: Fewer atomics: privatisation

IN THE EXAM In the block priority code of slide 7, the counter must be read as volatile or atomically, or the compiler may move the read out of the loop and spin for ever. CUDA does not promise which blocks are running or in what order they start. That this code usually works is luck.

Lesson: Priority locks and grid wide barriers

STEP 8 Tensor cores and fast matmul

8.1 Low precision numbers and quantisation

Float formats, range and detail, mixed precision, INT8 quantisation.

KEY IDEAS

- **The formula** x = (−1)^s × 1.m × 2^(e − bias), bias = 2^(E−1) − 1, gap after 1.0 = 2^−M.
- **Range against detail** Exponent bits give range, mantissa bits give detail. FP16 overflows at 65,504, BF16 keeps FP32 range with about 2 to 3 digits.
- **Which format** FP32 master weights, BF16 training, FP16 with loss scaling, TF32 for FP32 code on tensor cores, INT8 or FP8 for inference.
- **Wide accumulator** Narrow inputs, FP32 sums. In FP16, 2048 + 1 = 2048.
- **Quantisation** q = round(x ÷ s) + z, x̂ = s(q − z). Symmetric INT8: s = R ÷ 127. Error at most s ÷ 2 unless clipped.

FORMULAS

Fewer bytes, higher intensity Exam favourite

$$\text{AI} = \text{FLOPs} \div \text{bytes}$$

Halve the bytes and the arithmetic intensity doubles. Batch 1 FC layer: AI ≈ 1 in FP16, ≈ 2 with INT8 weights.

Lesson: Why use fewer bits?

Value of a normal number Exam favourite

$$x = (-1)^{s} \times 1.m_1 m_2 \ldots m_M \text{ (binary)} \times 2^{e\,-\,\text{bias}}$$

The leading 1 is not stored, so M bits give M + 1 bits of precision.

Lesson: How a float is stored, and the formats side by side

Bias Exam favourite

$$\text{bias} = 2^{E\,-\,1} - 1$$

5 bits: 15. 8 bits: 127. 11 bits: 1023.

Lesson: How a float is stored, and the formats side by side

Gap after 1.0

$$\varepsilon = 2^{-M}$$

FP32 1.2×10^{-7}, FP16 9.8×10^{-4}, BF16 7.8×10^{-3}.

Lesson: How a float is stored, and the formats side by side

Decimal digits

$$\approx (M + 1) \times \log_{10} 2 \approx 0.3\,(M + 1)$$

FP32 about 7, FP16 about 3, BF16 about 2.4.

Lesson: How a float is stored, and the formats side by side

Quantise

Exam favourite

$$q = \text{clip}(\text{round}(x \div s) + z,\ q_{min},\ q_{max})$$

s is the scale, z the zero point.

Lesson: Quantisation step by step

Bring back

Exam favourite

$$\hat{x} = s \times (q - z)$$

The rounding error is at most s ÷ 2 unless the value was clipped.

Lesson: Quantisation step by step

Symmetric INT8

$$s = R \div 127,\ z = 0$$

For values between −R and R.

Lesson: Quantisation step by step

Asymmetric (UINT8)

$$s = (\max - \min) \div 255,\ z = \text{round}(-\min \div s)$$

For values that are not centred on 0, such as ReLU outputs.

Lesson: Quantisation step by step

TABLES

Table: family · examples · time goes into · tensor cores help?

Family	Examples	Time goes into	Tensor cores help?
Math limited	convolution, fully connected, recurrent	calculating	a lot
Memory limited	bias add, normalisation, ReLU, dropout	moving numbers	not at all

Lesson: Why use fewer bits?

Table: format · e · m · largest

Format	E	M	Largest	Gap after 1.0	Digits
FP64	11	52	1.8×10^{308}	2.2×10^{-16}	16
FP32	8	23	3.4×10^{38}	1.2×10^{-7}	7.2
TF32	8	10	3.4×10^{38}	9.8×10^{-4}	3.3
FP16	5	10	65,504	9.8×10^{-4}	3.3
BF16	8	7	3.4×10^{38}	7.8×10^{-3}	2.4
FP8 E4M3	4	3	448	0.125	about 1
FP8 E5M2	5	2	57,344	0.25	under 1

Lesson: How a float is stored, and the formats side by side

Table: format · used for · watch out for

Format	Used for	Watch out for
FP32	master weights in training, science, anything delicate	slow on tensor cores
TF32	FP32 code on Ampere and later, about 8 times faster on an A100	not a storage format: memory still holds FP32
FP16	inference, training with loss scaling	tiny range: gradients underflow, 70,000 overflows
BF16	the default for training large models	only about 2 to 3 digits
FP8, FP4	Hopper and Blackwell training and inference	needs a scale factor per tensor or per block
INT8, INT4	inference after quantisation	needs a scale factor, accuracy must be checked

Lesson: Which format when, and why the sum stays wide

TIPS AND TRAPS

REMEMBER

Loss scaling for FP16 training: multiply the loss by a large number such as 1024 before back propagation so that small gradients do not round to 0, then divide the gradients by the same number. BF16 has the range of FP32 and does not need it.

Lesson: Which format when, and why the sum stays wide

IN THE EXAM

Slide 11's bars (TF32, FP16 and BF16 at 4 times, FP8 at 8, FP4 at 16) are labelled illustrative. On an A100, TF32 is 8 times and FP16 16 times the FP32 CUDA core rate. Always use the data sheet of the GPU in front of you.

Lesson: Which format when, and why the sum stays wide

8.2 Tensor cores, GEMM and WMMA

FC layer intensity, GEMM, im2col, tensor core peaks, WMMA, waves.

KEY IDEAS

- **Math limited test** AI = ops ÷ bytes > peak ÷ bandwidth. A100 FP16 ridge ≈ 201.
- **FC layer** AI = N_in N_out B ÷ (N_in B + N_in N_out + N_out B). 4096 layer: 4096B ÷ (4096 + 2B), math limited from batch 223.
- **GEMM** 2MNK FLOPs. Convolution: M = out channels, N = batch × output pixels, K = in channels × filter size (implicit GEMM).
- **Tensor core peak** SMs × TCs × FMAs per clock × 2 × clock. V100 125, A100 312, H100 989 TFLOP/s.
- **WMMA** One warp per 16 × 16 tile, fragments, load_matrix_sync, mma_sync (8192 FLOPs), store_matrix_sync. FP16 in, FP32 sum.
- **Waves** waves = ⌈blocks ÷ (SMs × blocks per SM)⌉, efficiency = n ÷ (W × waves). 217 blocks on 108 SMs: 67 percent.

FORMULAS

Time for maths and for data

$$T_{math} = ops \div BW_{math},\ T_{mem} = bytes \div BW_{mem}$$

With overlap, the larger one decides the speed.

Lesson: When is a layer limited by maths?

Math limited when Exam favourite

$$ops \div bytes > BW_{math} \div BW_{mem}$$

Left: the arithmetic intensity of the operation. Right: the ops per byte ratio of the GPU, the ridge point of the roofline.

Lesson: When is a layer limited by maths?

Work

$$\text{ops} = 2\, N_{in}\, N_{out}\, B$$

Each output is a dot product of length N_{in}: one multiply and one add per term.

Lesson: When is a layer limited by maths?

Bytes in FP16

$$\text{bytes} = 2\,(N_{in}\, B + N_{in}\, N_{out} + N_{out}\, B)$$

Read X, read W, write Y.

Lesson: When is a layer limited by maths?

4096 × 4096 layer Exam favourite

$$AI = 4096\, B \div (4096 + 2B)$$

Batch 1 gives about 1. The limit for a huge batch is 2048.

Lesson: When is a layer limited by maths?

Peak of the tensor cores Exam favourite

$$\text{peak} = \text{SMs} \times \text{tensor cores per SM} \times \text{FMAs per clock} \times 2 \times \text{clock}$$

An FMA counts as 2 FLOPs.

Lesson: Tensor cores and where their speed comes from

Warp granularity

$$\text{lane efficiency} = T \div (32\, \lceil T \div 32 \rceil)$$

33 threads need 2 warps: 51.6 percent of lanes used. 65 threads: 67.7 percent.

Lesson: Staircases: warp granularity and wave quantisation

Waves Exam favourite

$$\text{waves} = \lceil \text{blocks} \div (\text{blocks per SM} \times \text{SMs}) \rceil$$

One wave fills every SM once.

Lesson: Staircases: warp granularity and wave quantisation

Wave efficiency Exam favourite

$$n \div (W \times \lceil n \div W \rceil), \text{ with } W = \text{blocks per wave}$$

217 blocks on an A100: 217 ÷ 324 = 67 percent.

Lesson: Staircases: warp granularity and wave quantisation

TABLES

Table: layer · m · n · k

Layer	M	N	K
Fully connected	output features	batch	input features
Convolution (implicit GEMM)	output channels	batch × output height × output width	input channels × filter height × filter width
Attention	sequence	sequence or head size	head size or sequence

Lesson: Everything is a matrix multiply

Table: gpu · sms × tcs · fmas per clock · clock

GPU	SMs × TCs	FMAs per clock	Clock	Peak
V100 FP16	80 × 8	64	1.53 GHz	125 TFLOP/s
T4 FP16	40 × 8	64	1.59 GHz	65 TFLOP/s (FP32: 8.1)
A100 FP16 / BF16	108 × 4	256	1.41 GHz	312 TFLOP/s
A100 TF32	108 × 4	128	1.41 GHz	156 TFLOP/s (FP32 CUDA cores: 19.5)
H100 SXM BF16	132 × 4	512	1.83 GHz	989 TFLOP/s

Lesson: Tensor cores and where their speed comes from

Table: level · what you write

Level	What you write
Libraries	cuBLAS (cublasGemmEx), cuDNN, PyTorch with FP16, BF16 or allow_tf32
Templates	CUTLASS and CuTe
WMMA	the API above: portable and simple
PTX mma.sync	warp level with exact register layouts (Ampere)
PTX wgmma	Hopper: 4 warps issue asynchronous tensor core work fed by TMA

Lesson: Programming tensor cores with WMMA

TIPS AND TRAPS

REMEMBER — im2col copies each pixel into up to 9 rows, so the matrix is 9 times bigger than the picture. Libraries such as cuDNN never build it in memory. They compute the addresses on the fly, which is why it is called implicit GEMM.

Lesson: Everything is a matrix multiply

REMEMBER — Real kernels rarely see the data sheet peak. Under a heavy matmul an H100 hits its 700 W power limit and lowers its clock, so 750 TFLOP/s may be closer to what is reachable than 750 ÷ 989 = 76 percent suggests.

Lesson: Tensor cores and where their speed comes from

IN THE EXAM — Slide 16 says shared memory is used for the matrix C. In real kernels such as CUTLASS, C stays in registers during the main loop and only passes through shared memory when it is written out at the end.

Lesson: Programming tensor cores with WMMA

REMEMBER — Fixes: sizes that are multiples of 32 threads and of the wave size, a tile size that divides the problem evenly, persistent kernels, or stream-K, which splits the leftover work along K across the idle SMs (as CUTLASS does). The same effect exists on AMD GPUs.

Lesson: Staircases: warp granularity and wave quantisation

8.3 How fast matmul kernels are built

H100 peak, naive kernel, tile intensity, swizzling, wgmma, pipelines, the ladder.

KEY IDEAS

- **Speed of light** H100 SXM BF16: 132 × 4 tensor cores × 1024 FLOPs × 1.83 GHz ≈ 989 TFLOP/s. Power throttling lowers the real ceiling.
- **Naive kernel** Coalesced order 3171 GFLOP/s (43 ms), swapped order 243 GFLOP/s (565 ms). 13 times from one index.
- **Tile intensity** AI = (2 ÷ b) B_M B_N ÷ (B_M + B_N). Square is best. FP32 T × T: T ÷ 4. BF16: T ÷ 2.
- **Hopper** TMA copies tiles, swizzle slot = chunk XOR row, wgmma on 4 warps, producer and consumer warp groups.
- **Ladder** 32 (no tensor cores) to 317 (tensor cores + TMA) to 764 TFLOP/s, about 77 percent of peak.

FORMULAS

Intensity of a BM × BN tile

Exam favourite

$$AI = (2 \div b) \times B_M B_N \div (B_M + B_N)$$

b = bytes per number. BK cancels, so only the tile shape matters.

Lesson: Tiles: more work per byte

Square is best

$$B_M + B_N \geq 2\sqrt{(B_M B_N)}$$

For a fixed area the sum is smallest when $B_M = B_N$ (AM-GM).

Lesson: Tiles: more work per byte

Square FP32 tile T × T

Exam favourite

$$AI = T \div 4$$

32 × 32: 8. 128 × 128: 32. BF16 doubles these: AI = T ÷ 2.

Lesson: Tiles: more work per byte

TABLES

Table: part · h100 sxm5

Part	H100 SXM5
SMs	132 (the full chip has 144, some are switched off)
Each SM	4 quadrants, each with a warp scheduler, FP32 units, a tensor core and load and store units
Warps issuing per clock	4 per SM (one per quadrant), out of up to 64 resident
Registers	64K 32 bit registers per SM
Shared memory and L1	up to 228 KB per SM, the same SRAM split by configuration
L2	50 MB, shared by all SMs
HBM3	80 GB at about 3.35 TB/s

Lesson: The H100 and its speed of light

Table: kernel · measured on h100 · time for 2 × 4096³ flops

Kernel	Measured on H100	Time for 2 × 4096³ FLOPs
x feeds the column (coalesced)	3171 GFLOP/s	43 ms
x feeds the row (uncoalesced)	243 GFLOP/s	565 ms, 13 times slower

Lesson: The naive kernel and a 13 times lesson

Table: trick · what it fixes

Trick	What it fixes
Persistent kernel	one long lived block per SM loops over tiles, so stores overlap the next loads and no wave is half empty
Hilbert curve order of tiles	neighbouring SMs use the same strips of A and B, so L2 hits go up
Thread block clusters and multicast	2 SMs share loaded tiles through distributed shared memory
Two consumer warp groups	a 128 × 256 tile without running out of registers

Lesson: Hopper tricks: TMA, swizzling, wgmma, pipelines

Table: course idea · where it shows up in the ladder

Course idea	Where it shows up in the ladder
Coalescing (3.2)	the 13 times gap of the naive kernel
Shared memory and banks (3.3)	tiles and swizzling
Tiled matmul (5.2)	block tiles, warp tiles, thread tiles
Occupancy and waves (2.4, 8.2)	persistent kernels
Streams and overlap (6.2)	producer and consumer pipelines
Barriers (7.2)	mbarrier and faster barriers
Tensor cores (8.2)	the 10 times jump

Lesson: The performance ladder

TIPS AND TRAPS

REMEMBER

Under a heavy matmul the H100 reaches its 700 W power limit and lowers its clock. At 1.5 GHz the peak is 989 × 1.5 ÷ 1.83 ≈ 811 TFLOP/s, so a kernel at 750 TFLOP/s is at about 92.5 percent of what is reachable.

Lesson: The H100 and its speed of light

IN THE EXAM

With the numbers used here (756 TFLOP/s BF16 on the PCIe card, 2.0 TB/s) the ridge is about 378 FLOP/byte. The blog quotes about 410 with slightly different specs. Either way a kernel needs hundreds of FLOPs per DRAM byte, so tiles must also be reused through L2 and shared memory.

Lesson: Tiles: more work per byte

STEP 9 Tools and Python on the GPU

9.1 Debugging and profiling CUDA code

Error checks, compute-sanitizer, cuda-gdb, Nsight, PTX and SASS.

KEY IDEAS

- **Order of tools** Check every call. Wrong: compute-sanitizer, printf, cuda-gdb. Slow: nsys, then ncu on the hot kernel.
- **Sanitizer tools** memcheck (bounds), racecheck (shared memory races), initcheck (uninitialised reads), synccheck (bad barriers).
- **cuda-gdb** Build with -g -G. set cuda break_on_launch application, cuda block (1,0,0) thread (31,0,0), p x.
- **Reading a report** Time = cycles ÷ clock. SM active ÷ elapsed shows idle SMs. Both units under 60 percent: latency bound, read the stall reasons.
- **PTX and SASS** PTX is virtual and portable (%ctaid.x, ld.global), SASS is the real code (S2R, LDG, FADD, STG). See them with nvcc -ptx, cuobjdump -sass.

TABLES

Table: tool · finds · example in this course

Tool	Finds	Example in this course
memcheck (default)	out of bounds or misaligned accesses, bad frees, leaks	badAccess above
racecheck	shared memory races (a missing __syncthreads)	unsafe_inc in chapter 7.2
initcheck	reads of global memory that was never written	reading cudaMalloc memory before setting it
synccheck	illegal barriers (a __syncthreads inside a divergent if)	the reduction in chapter 5.1

Lesson: compute-sanitizer

Table: tool · shows · use it

Tool	Shows	Use it
Nsight Systems (nsys)	the whole program timeline: CPU, API calls, copies, kernels per stream, NVTX ranges	first: which kernel or copy takes the time?
Nsight Compute (ncu)	one kernel in depth: throughput, occupancy, traffic per memory level, stall reasons, roofline	second: why is that kernel slow?
compute-sanitizer	correctness	for wrong results, not speed
nvvp, nvprof	the old profilers	not supported on compute capability 8.0 and newer

Lesson: Profiling with Nsight Systems and Nsight Compute

Table: metric · value · meaning

Metric	Value	Meaning
SM frequency	880 MHz	the profiler locks the clock to the base value
Elapsed cycles	10,633	10,633 ÷ 880 MHz = 12.08 µs, matching the duration
Duration	12.06 µs	a tiny kernel: a launch costs about 5 µs
Memory throughput	44.35 %	the busiest memory unit at 44 % of its peak
DRAM throughput	42.92 %	of the T4's 320 GB/s
SM active cycles	8,132	8132 ÷ 10,633 = 76 %: SMs idle a quarter of the time
Compute (SM) throughput	36.98 %	the busiest compute pipe at 37 %

Lesson: Profiling with Nsight Systems and Nsight Compute

Table: sass · meaning

SASS	Meaning
S2R R0, SR_CTAID.X	read a special register (blockIdx.x)
IMAD	integer multiply add, for example the global index or an address
LDG.E	load from global memory with a 64 bit address
FADD, FFMA	float add, fused multiply add
STG.E	store to global memory
ISETP, @P0 BRA	compare into a predicate, then a branch taken only where the predicate holds

Lesson: Reading PTX and SASS

TIPS AND TRAPS

COMMON MISTAKE This program usually runs without any message. The stray writes land in memory the program happens to own, so nothing crashes. That is exactly why out of bounds bugs are dangerous and why the sanitizer comes first.

Lesson: A workflow for wrong or slow code

IN THE EXAM Slide 6 says threads 0 to 21 write out of bounds. Thread i writes data[i + 10], so even thread 0 writes data[10], one past the end. All 32 threads write out of bounds, to indices 10 to 41.

Lesson: compute-sanitizer

REMEMBER Lecture 7 showed the PTX of the naive matmul: two ld.global loads for every mad. Global loads cost hundreds of cycles, the multiply add only a few. That is the case for tiling and shared memory, read straight from the compiler output.

Lesson: Reading PTX and SASS

9.2 GPU programming from Python

Numba, CuPy, Triton and PyTorch, mapped to CUDA C.

KEY IDEAS

- **Routes** CuPy for NumPy code, Numba for kernels in Python, Triton for block level kernels, PyTorch for deep learning. All compile just in time.
- **Numba twins** @cuda.jit, cuda.grid(1), to_device, device_array_like, add[blocks, threads], copy_to_host, shared.array, syncthreads, atomic.add.
- **Simulator** NUMBA_ENABLE_CUDASIM=1 runs kernels on the CPU for testing and debugging.
- **CuPy** Each operation is a kernel and a full memory pass. Fuse with ElementwiseKernel or cp.fuse. RawKernel for CUDA C.
- **Triton and PyTorch** Triton: program_id, offs, mask, tl.load and tl.store. PyTorch: pin_memory, non_blocking, Stream, synchronize.

TABLES

Table: library · you write · good for

Library	You write	Good for
CuPy	NumPy style arrays on the GPU, plus element wise and raw CUDA kernels	speeding up NumPy code with almost no change
Numba	kernels in Python with @cuda.jit, cuda.grid, shared arrays, atomics	learning CUDA from Python, custom kernels
PyCUDA	CUDA C source strings, driver level control	full control from Python
Triton	block level kernels, the compiler handles threads and tiling	fast custom deep learning kernels (used by PyTorch)
PyTorch	tensors, torch.cuda streams, pinned memory	deep learning, and most real GPU work in Python

Lesson: Five ways to reach the GPU from Python

Table: cuda c · numba

CUDA C	Numba
__global__ void add(...)	@cuda.jit above a Python function
blockIdx.x * blockDim.x + threadIdx.x	cuda.grid(1)
cudaMalloc + cudaMemcpy host to device	cuda.to_device(a)
cudaMalloc only	cuda.device_array_like(a)
add<<<>>>(...)	add[blocks, threads](...)
cudaMemcpy device to host	d_c.copy_to_host()
__shared__ float s[256]	cuda.shared.array(256, dtype=float32)
__syncthreads()	cuda.syncthreads()
atomicAdd(&out[0], v)	cuda.atomic.add(out, 0, v)

Lesson: Numba: CUDA kernels written in Python

Table: cuda idea · pytorch

CUDA idea	PyTorch
cudaMallocHost	tensor.pin_memory()
cudaMemcpyAsync	tensor.to("cuda", non_blocking=True)
cudaStream_t	torch.cuda.Stream()
cudaDeviceSynchronize	torch.cuda.synchronize()
TF32 tensor cores	torch.backends.cuda.matmul.allow_tf32 = True
custom kernel	torch.compile (generates Triton) or a C++ extension

Lesson: PyTorch: streams, pinned memory and tensor cores

TIPS AND TRAPS

REMEMBER The simulator runs each GPU thread as a Python thread on the CPU. It is very slow but lets you test and debug kernels, including with print and pdb, on any laptop. The listings above were checked this way.

Lesson: Numba: CUDA kernels written in Python

REMEMBER Every separate operation reads and writes the whole array. 2 * x + 1 is two passes over memory. cp.fuse or an ElementwiseKernel does it in one pass, which matters because element wise work is memory bound.

Lesson: CuPy: NumPy on the GPU

STEP 10 Exam practice

10.1 Quiz 1 solved step by step

Quiz 1: all ten answers with steps, traps and variants.

KEY IDEAS

- **Q1** Use the statement about the SM: 4 FMA × 2 × 1.5 GHz = 12 GFLOPS, option 2.
- **Q2 and Q3** Naive: $B^2 \times 2N$ loads per block (524,288). Tiled: $B^2 \times 2 \times N/B$ (32,768).
- **Q4** No __syncthreads: both races possible, answer 3. Fix with one barrier after loading and one after computing.
- **Q5 and Q10** Column store with stride 2048 floats: 32 segments. Warps = blocks × threads per block ÷ 32 = 32,768.
- **Q6 to Q9** __global__ void, y[2] = 13, global memory, tids 1 to 29 step 4 sum to 120.

TABLES

Table: q · topic · answer · chapter

Q	Topic	Answer	Chapter
1	peak FP32 throughput of one SM	2 (12 GFLOPS)	1.1, 4.2
2	global loads per block, naive matmul	524,288	5.2
3	global loads per block, tiled matmul	32,768	5.2
4	races in a tiled kernel without barriers	3 (both true)	5.2, 7.2
5	128 byte segments for a transposed store	32	3.2
6	return type of a kernel	5 (__global__ void)	1.4
7	trace a matrix vector kernel	13	2.1
8	memory visible to all SMs for the whole run	1 (global memory)	3.1
9	sum of thread ids in a divergent branch	120	6.1
10	warps launched	32,768	2.3

Lesson: The paper and the answers

Table: qualifier · runs on · called from

Qualifier	Runs on	Called from
__global__	GPU	CPU (a launch), must return void
__device__	GPU	GPU code only, any return type
__host__	CPU	CPU (the default)
__shared__	not a function: a variable in shared memory	

Lesson: Q1, Q6 and Q8: peak, kernels and memory

TIPS AND TRAPS

IN THE EXAM The other reading, 4 FMA per core, gives 64 × 4 × 2 × 1.5 = 768 GFLOPS per SM, which matches no option. When a question gives two numbers for the same thing, use the one that describes exactly what is asked (here the SM) and check the options.

Lesson: Q1, Q6 and Q8: peak, kernels and memory

IN THE EXAM Answer 3: both statements are true for the quiz code, because it has no __syncthreads. The fixed version above needs one barrier after loading and one after computing.

Lesson: Q2 to Q4: loads and races in matrix multiply

10.2 Challenge problems

Hard multi step problems with full solutions.

KEY IDEAS

- **Batch threshold** n × n layer: AI = nB ÷ (n + 2B). A100 4096: 223. H100 4096: 345. A100 1024: 330.
- **Occupancy** Blocks = min(registers, threads, block limit, shared memory). Registers bind at 40 × 256 on an A100: 75 percent.
- **Banks** Stride k gives a gcd(k, 32) way conflict. Pad rows to 33 for column reads.
- **Scan work** Hillis and Steele n log n − (n − 1) in log n steps. Blelloch 2(n − 1) in 2 log n steps.
- **Tile budget** Double buffered FP32 with BK = 32 in 48 KB: BM + BN ≤ 192, best 96 × 96, AI 24.

FORMULAS

Conflict degree for stride k Exam favourite

$$\text{degree} = \gcd(k, 32)$$

Thread t goes to bank (t k) mod 32.

Lesson: Problem 3: banks, strides and padding

Hillis and Steele

$$\text{adds} = n \log_2 n - (n - 1), \text{steps} = \log_2 n$$

Step d updates $n - 2^d$ elements.

Lesson: Problem 4: work against steps in scan

Blelloch Exam favourite

$$\text{adds} = 2\,(n - 1), \text{steps} = 2 \log_2 n$$

Up sweep then down sweep.

Lesson: Problem 4: work against steps in scan

10.3 Exam revision

Formula sheets, slide corrections, traps and mixed problems.

KEY IDEAS

- **Three formula groups** Speed and limits (Amdahl, peak, roofline), threads and memory (index, warps, occupancy, banks, tiling), patterns (reduction, scan, pipeline, formats).
- **Write the unit** Threads or warps, per thread or per block, bytes read and written, FMA = 2 FLOPs, GB = 10^9 bytes.
- **Slide corrections** Thread 343 is (3, 3, 0). Heat centre 0.4. Width³ loads. Sharpen gives 30. Attempt 2 breaks mutual exclusion. C tile in registers.
- **T4 numbers** 40 SMs, 8.14 TFLOP/s FP32, 320 GB/s, ridge 25.4, 1024 threads and 16 blocks per SM.
- **Next step rule** Memory bound: reuse data (tiles). Copy bound: keep data on the GPU or overlap. Latency bound: more warps or more independent work.

FORMULAS

Speedup and efficiency Exam favourite

$$S = T_{old} \div T_{new}, \quad E = S \div N$$

Count copy in, kernel and copy out in T_{new}.

Lesson: Formula sheet 1: speed and limits

Amdahl (fixed problem) Exam favourite

$$S(N) = 1 \div ((1 - P) + P \div N), \quad \text{ceiling } 1 \div (1 - P)$$

P is the parallel fraction of the old time.

Lesson: Formula sheet 1: speed and limits

Gustafson (problem grows)

$$S(N) = (1 - P) + P\,N$$

Weak scaling: the parallel work grows with N.

Lesson: Formula sheet 1: speed and limits

Peak FLOP/s Exam favourite

$$\text{peak} = \text{SMs} \times \text{cores per SM} \times 2 \times \text{clock}$$

The 2 is one FMA per core per cycle.

Lesson: Formula sheet 1: speed and limits

Little's law

$$\text{in flight} = \text{latency} \times \text{throughput}$$

Warps needed = in flight ÷ 32. Bytes in flight = bandwidth × latency.

Lesson: Formula sheet 1: speed and limits

Arithmetic intensity Exam favourite

$$AI = \text{FLOPs} \div \text{bytes moved}$$

Count bytes read and written.

Lesson: Formula sheet 1: speed and limits

Roofline

Exam favourite

$$\text{attainable} = \min(\text{peak}, \text{AI} \times \text{bandwidth}), \quad \text{ridge} = \text{peak} \div \text{bandwidth}$$

Left of the ridge: memory bound. Right: compute bound.

Lesson: Formula sheet 1: speed and limits

Time bound

$$t \geq \max(\text{FLOPs} \div \text{peak}, \text{bytes} \div \text{bandwidth})$$

The larger term tells you what to fix.

Lesson: Formula sheet 1: speed and limits

Global index and blocks

Exam favourite

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}, \quad \text{blocks} = \lceil N \div B \rceil$$

Guard with $i < N$. Idle threads = blocks × B − N.

Lesson: Formula sheet 2: threads and memory

Going backwards

$$\text{block} = \lfloor g \div (B_x B_y) \rfloor, \; x = r \bmod B_x, \; y = \lfloor r \div B_x \rfloor$$

r is the leftover g − block × $B_x B_y$.

Lesson: Formula sheet 2: threads and memory

Warps

Exam favourite

$$\text{warps per block} = \lceil B \div 32 \rceil, \quad \text{lane} = t \bmod 32$$

$t = x + y\,B_x + z\,B_x B_y$, x moves fastest.

Lesson: Formula sheet 2: threads and memory

Occupancy

Exam favourite

$$\text{blocks per SM} = \min(\text{threads, blocks, registers, shared memory limits})$$

Occupancy = active warps ÷ max warps per SM.

Lesson: Formula sheet 2: threads and memory

Coalescing

$$\text{sectors per warp load} = \text{distinct 32 byte pieces touched}$$

Floats: stride 1 gives 4 sectors, stride 8 or more gives 32.

Lesson: Formula sheet 2: threads and memory

Bank conflicts

Exam favourite

$$\text{degree} = \gcd(\text{stride}, 32)$$

Same word for all lanes is a broadcast. Pad rows to 33.

Lesson: Formula sheet 2: threads and memory

Matmul loads

Exam favourite

$$\text{naive } 2N^3, \quad \text{tiled } 2N^3 \div T$$

Per thread 2N against 2N ÷ T. Tile AI in FP32 is T ÷ 4.

Lesson: Formula sheet 2: threads and memory

Halo tiles

$$\text{loads per output} = (B + 2R)^d \div B^d$$

d is the number of dimensions, R the radius.

Lesson: Formula sheet 2: threads and memory

Reduction tree

Exam favourite

$$\log_2 B \text{ levels}, \; B - 1 \text{ additions}, \; 1 + \log_2 B \text{ barriers}$$

One atomic per block joins the blocks.

Lesson: Formula sheet 3: patterns, concurrency and number formats

Scans

Exam favourite

$$\text{Hillis and Steele } n \log_2 n - (n - 1) \text{ adds in } \log_2 n \text{ steps, Blelloch } 2(n - 1) \text{ adds in } 2 \log_2 n \text{ steps}$$

Blelloch does less work, Hillis and Steele has fewer steps.

Lesson: Formula sheet 3: patterns, concurrency and number formats

Divergence

$$\text{efficiency} = \text{busy lane ticks} \div (32 \times \text{total ticks})$$

Paths inside one warp run one after the other.

Lesson: Formula sheet 3: patterns, concurrency and number formats

Pipeline

Exam favourite

$$T = (a + b + c) + (n - 1) \times \max(a, b, c)$$

Equal stages give speedup $3n \div (n + 2)$.

Lesson: Formula sheet 3: patterns, concurrency and number formats

Waves

$$\text{waves} = \lceil \text{blocks} \div (\text{SMs} \times \text{blocks per SM}) \rceil$$

A last wave that is almost empty wastes most of the GPU.

Lesson: Formula sheet 3: patterns, concurrency and number formats

Float value

Exam favourite

$$x = (-1)^{s} \times 1.m \times 2^{e - \text{bias}}, \quad \text{bias} = 2^{E-1} - 1$$

FP32 1/8/23, FP16 1/5/10, BF16 1/8/7, TF32 1/8/10.

Lesson: Formula sheet 3: patterns, concurrency and number formats

Symmetric INT8

$$s = \max|x| \div 127, \quad q = \text{round}(x \div s)$$

Error at most s ÷ 2 unless a value is clipped.

Lesson: Formula sheet 3: patterns, concurrency and number formats

Interleavings

$$(n\ s)! \div (s!)^{n}$$

n threads with s steps each. Two threads of 3 steps: 20 orders.

Lesson: Formula sheet 3: patterns, concurrency and number formats

TABLES

Table: day · what to do · chapters

Day	What to do	Chapters
1	Speed and limits: speedup, Amdahl, peak, roofline. Redo the formula cards below from memory.	1.1 to 1.3, 4.2
2	Threads: indexing forwards and backwards, warps, occupancy. Do the launch problem below by hand.	2.1 to 2.4
3	Memory: hierarchy, coalescing, banks, tiling. Draw one warp and count sectors.	3.1 to 3.3, 5.2
4	Patterns: reduction, stencils, convolution, scan, histogram.	5.1 to 5.5
5	Concurrency: divergence, streams, races, locks, barriers, atomics.	6.1 to 7.2
6	Modern GPUs: number formats, tensor cores, fast matmul, tools.	4.1, 8.1 to 9.2
7	Quiz 1 again under time, then three challenge problems.	10.1, 10.2

Lesson: How to revise in one week

Table: where · the slide or class said · what is right · chapter

Where	The slide or class said	What is right	Chapter
Lecture 3, slide 16	Global thread 343 has threadIdx (2, 3, 0)	It is (3, 3, 0). (2, 3, 0) is thread 342.	2.2
Lecture 3, mapping	Grids sit inside cores	A thread runs on a core lane, a block on one SM, a grid across the GPU.	2.3
Lecture 4, slide 11	Asynchrony is risky because kernels overwrite each other	Work in one stream runs in order. The risk is the CPU reading before the GPU is done.	6.2
Lecture 5 notes	1.00101 in binary = 1.5625	1.00101 in binary = 1.15625 = 37 ÷ 32.	4.1
Lecture 6, slide 14	Heat weights 0.25 centre and 0.1 for 6 neighbours	They add to 0.85, so heat leaks away. The centre must be 0.4.	5.3
Lecture 7, slide 6	Threads 0 to 21 write out of bounds	All 32 threads write out of bounds (indices 10 to 41).	9.1
Lecture 7, slide 8	The plain matmul reads Width² values	Each of the Width² threads reads 2 × Width, so 2 × Width³ in total.	5.2
Lecture 8	Sharpening pixel 30 gives 35	5 × 30 − (25 + 35 + 20 + 40) = 30.	5.4
Lecture 9, slide 11	Bars show exact format speeds	The bars are illustrative. TF32 is a compute mode, not a storage format.	8.1
Lecture 9, slide 16	The C tile lives in shared memory	In real kernels the C tile stays in registers. Only the epilogue uses shared memory.	8.2
Lecture 9 blog numbers	Ridge about 410	With the numbers used in class the ridge is about 378.	8.3
Lecture 10, slide 8	Attempt 2 deadlocks	Attempt 2 breaks mutual exclusion. Attempt 3 is the one that deadlocks.	7.1
Lecture 10, class	Peterson does not solve the problem	Peterson is correct when memory is in order, so on a GPU it needs fences.	7.1
Lecture 11 code	critical_kernel is safe as written	It can hang before Volta and needs __threadfence. The priority counter skips levels.	7.2

Lesson: Every slide correction in one table

Table: trap · do this instead

Trap	Do this instead
Counting threads when the question asks for warps	Divide by 32 and round up per block, then multiply by blocks.
Loads per thread when the question says per block	Multiply by threads per block. Quiz 1 question 2: 256 × 2048 = 524,288.
Forgetting the write in bytes moved	SAXPY reads x and y and writes y: 12 bytes per element.
Taking FMA as 1 FLOP	One FMA is 2 FLOPs, a multiply and an add.
Putting __syncthreads inside a divergent if	Every thread of the block must reach every barrier.
Treating occupancy as the goal	Enough warps to hide latency is enough. Measure, do not maximise.
Reading GB as 2^{30} bytes in bandwidth	Bandwidth uses 10^9 bytes per GB.
Forgetting the reserved 1 KB of shared memory per block	On CC 8.x, blocks = ⌊pool ÷ (bytes + 1 KB)⌋.
Thinking divergence happens across warps	Different warps may take different paths for free. Only lanes inside one warp diverge.
Amdahl with N where the question gives a speedup of a part	Use 1 ÷ ((1 − p) + p ÷ s) with s the speedup of that part.

Lesson: The traps that cost marks

TIPS AND TRAPS

IN THE EXAM

Most lost marks come from counting the wrong thing: threads instead of warps, loads per thread instead of per block, bytes one way instead of both. Write the unit next to every number.

Lesson: How to revise in one week

IN THE EXAM

If an exam question copies one of these slides, answer with the right value and add one line saying why. Markers reward the reasoning.

Lesson: Every slide correction in one table